ESCUELA POLITÉCNICA SUPERIOR DE CÓRDOBA
Universidad de Córdoba

Introducción al Aprendizaje Automático

Grado en Ingeniería Informática

Memoria de Prácticas

Autores: Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Abril, 2026

UNIVERSIDAD DE CÓRDOBA

Dedicado a
mi familia

Resumen

Esta memoria documenta el desarrollo y los resultados obtenidos en las sesiones experimentales de la asignatura Introducción al Aprendizaje Automático. El documento abarca el análisis empírico de algoritmos fundamentales, comenzando por la optimización mediante descenso por gradiente, donde se explora el impacto de la tasa de aprendizaje en la convergencia, la eficiencia computacional de los mini-batches y la implementación de criterios de parada. Posteriormente, se estudia el protocolo de Validación Cruzada (K-Fold) para la selección de modelos, así como las técnicas de regularización Ridge (L2) y Lasso (L1) para prevenir el sobreajuste y facilitar la selección automática de atributos.

Palabras clave: Aprendizaje Automático, Descenso por Gradiente, Validación Cruzada, Regularización, Ridge, Lasso.

Abstract

This report documents the development and results obtained during the experimental sessions of the course Introducción al Aprendizaje Automático. The document covers the empirical analysis of fundamental algorithms, starting with gradient descent optimization, where the impact of the learning rate on convergence, the computational efficiency of mini-batches, and the implementation of tolerance-based stopping criteria are explored. Subsequently, the K-Fold Cross-Validation protocol for model selection is studied, along with Ridge (L2) and Lasso (L1) regularization techniques to prevent overfitting and enable automatic feature selection.

Keywords: Machine Learning, Gradient Descent, Cross-Validation, Regularization, Ridge, Lasso.

Índice general

Resumen **II**

Abstract **III**

Índice de figuras **VII**

Índice de tablas **VIII**

1. Práctica 1: La Física del AA **1**
1.1. Tarea 1: Tasa de aprendizaje 1
1.1.1. Escenario A (α_{slow}): convergencia lenta 1
1.1.2. Escenario B (α_{opt}): "curva de codo" 2
1.1.3. Escenario C (α_{osc}): oscilación amortiguada 3
1.1.4. Escenario D (α_{fail}): divergencia 4
1.2. Tarea 2: Mini-batch 6
1.2.1. Batch completo 6
1.2.2. Mini-batch (32 o 16) 7
1.2.3. Estocástico puro (batch = 1) 8
1.2.4. Tiempo de ejecución — Estocástico puro (batch = 1) 9
1.2.5. Tiempo de ejecución — Mini-batch (batch = 32) 9
1.2.6. Tiempo de ejecución — Batch completo (batch = 500) 10
1.3. Reto: Criterio de parada 10
1.4. Conclusiones 10

2. Práctica 2: Selección de Modelos **12**
2.1. Tarea 1: Estabilidad K-Fold 12
2.1.1. Experimento A — K = 2 (Validación simple): variabilidad entre ejecuciones 12
2.1.2. Experimento B — K = 10 (estándar): RMSE más estable . . . 13
2.1.3. Experimento C — K = 100 (Leave-One-Out): coste vs. precisión 14
2.1.4. Resumen comparativo (K = 2 vs 10 vs 100) 15
2.2. Tarea 2: Ridge vs. Lasso 16
2.2.1. Ridge (L2) — λ = 0,1, 10, 100 16
2.2.2. Lasso (L1) — λ = 0,1, 10, 100 17
2.2.3. El dilema de la complejidad — λ demasiado alto 19

2.3. Reto — El "Cazador de Variables" 20
2.4. Conclusiones 21

3. Práctica 3: Metodología y Rigor 22
3.1. Tarea 1: Lotería de la partición 22
3.1.1. Análisis estadístico de la partición 23
3.1.2. Registro de resultados (5 particiones aleatorias) 23
3.2. Tarea 2: Partición estratificada 25
3.2.1. Registro por folds (`StratifiedKFold`, $k = 10$) 25
3.3. Tarea 3: Comparativa de varianza 26
3.3.1. Resumen de resultados (10 iteraciones, 10 folds) 26
3.4. Tarea 4: Data Leakage 27
3.4.1. Comparativa: con y sin variable trampa 27
3.5. Reto: Auditoría Metodológica 28
3.6. Conclusión final 29

4. Práctica 4: Clasificación y Matriz de Costes 30
4.1. Tarea 1: Umbral por defecto 30
4.1.1. Modelo utilizado 30
4.1.2. Matriz de confusión 31
4.1.3. Cálculo del coste total 31
4.1.4. Interpretación 31
4.2. Tarea 2: Probabilidades y Umbrales 32
4.2.1. Análisis de distintos umbrales 32
4.2.2. Observaciones 32
4.3. Tarea 3: Optimización del coste 33
4.3.1. Gráfica de coste frente a umbral 33
4.3.2. Umbral óptimo 34
4.3.3. Interpretación 34
4.4. Tarea 4: Curvas ROC y PR 34
4.4.1. Curva ROC 34
4.4.2. Curva Precision-Recall 35
4.4.3. Discusión 35
4.5. Decisión final de ingeniería 35
4.6. Conclusión final 36

5. Práctica 5: Perceptrón Multicapa 37
5.1. Tarea 1: Barrera de linealidad 37
5.1.1. Resultados y Análisis Estructural 38
5.2. Tarea 2: Capacidad capa oculta 38

5.3. Tarea 3: ReLU vs. Sigmoide 40
5.3.1. Fronteras de decisión 41
5.3.2. Curvas de pérdida y velocidad de convergencia 41
5.4. Tarea 4: Análisis en Dígitos 42
5.4.1. Análisis de sobreajuste 43
5.4.2. Elección del modelo 43

6. Práctica 7: Modelos de Ensambles 46
6.1. Tarea 1: Árbol Solitario 46
6.2. Tarea 2: Random Forest 47
6.3. Tarea 3: Boosting 47
6.4. Comparación y Reflexión 48

Bibliografía 50

Índice de figuras

1.1. Tiempo de ejecución con batch=1. 9
1.2. Tiempo de ejecución con batch=32. 9
1.3. Tiempo de ejecución con batch=500. 10
1.4. Tiempo de convergencia interrumpida en la época 55 sujeto a $|J_t - J_{t-1}| < 10^{-5}$ para $\alpha = 0{,}01$ y batch= 32. 10

3.1. Variabilidad del número de ejemplos positivos en el conjunto de test según la semilla. La línea roja indica el valor esperado teórico (4). . . 24
3.2. Proporción de la clase positiva por fold con `StratifiedKFold`. La línea roja indica la proporción global del dataset (2 %). 25
3.3. Distribución de Accuracy, F1-macro y AUC-ROC a lo largo de 10 iteraciones de validación cruzada (10 *folds*). Cada métrica se muestra como un diagrama de cajas (*boxplot*). `StratifiedKFold` produce cajas más estrechas en las tres métricas, evidenciando menor varianza y mayor estabilidad en la evaluación. 26
3.4. Efecto del *data leakage*. Las barras rojas corresponden al escenario con la variable trampa; las verdes, al escenario real sin ella. 28

4.1. Matriz de confusión para el umbral estándar $\tau = 0{,}5$. 31
4.2. Coste total (€) en función del umbral de decisión. Como se aprecia en la Figura 4.2, el punto óptimo se alcanza en $\tau^* = 0{,}05$. 33
4.3. Curva ROC del clasificador. AUC-ROC = 0,8442. 34
4.4. Curva Precision-Recall del clasificador. Average Precision = 0,6551. . 35

5.1. Evaluación del rendimiento en un perceptrón de una sola capa. 38
5.2. Evolución de la frontera de decisión según la densidad de la red. . . . 39
5.3. Comparación visual de la frontera de decisión según la activación. . . 41
5.4. Comparativa de la velocidad de aprendizaje. 41
5.5. Matriz de confusión de la arquitectura **MLP(20)**. El desempeño es casi impecable en todas las clases; los escasos errores se concentran en pares de dígitos visualmente similares. 44

Índice de tablas

3.1. Comparativa de estabilidad de métricas: KFold estándar vs. StratifiedKFold (10 iteraciones × 10 folds). 26

4.1. Resumen de FP, FN y Coste Total para distintos umbrales de decisión. 32

5.1. Resultados experimentales según la arquitectura. 39
5.2. Métricas comparativas por función de activación (arquitectura `(20,)`). 42
5.3. Métricas obtenidas para distintas arquitecturas en el dataset de dígitos. 43

6.1. Comparativa global de los modelos evaluados. 48

Cuaderno de Laboratorio — Práctica 1: La Física del Aprendizaje Automático

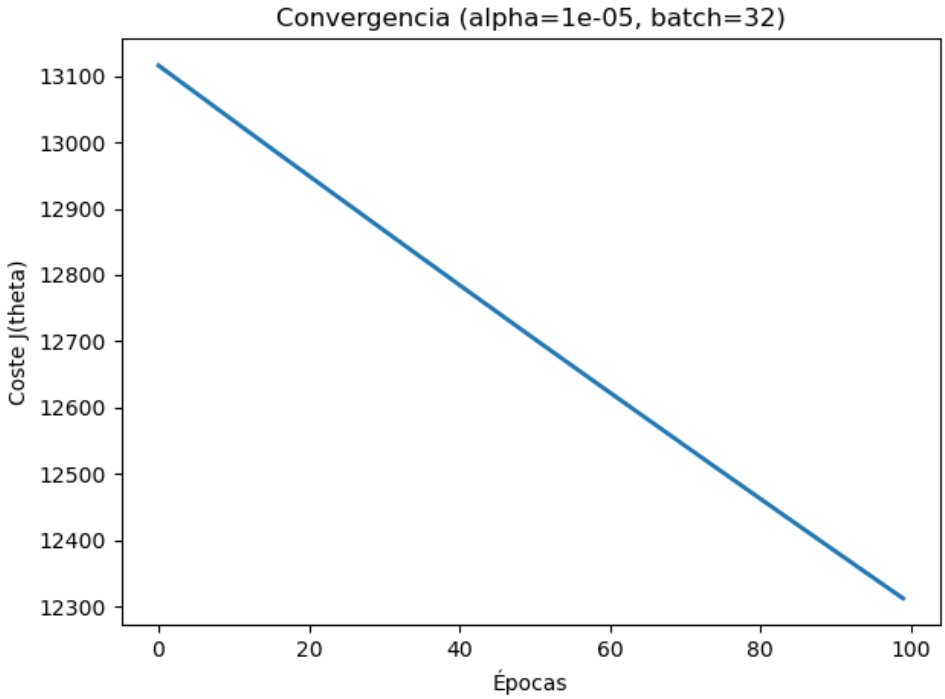
Objetivo

Comprender la dinámica de la optimización mediante gradiente descendente y el impacto de los hiperparámetros en la convergencia del algoritmo.

1.1. Tarea 1 — Importancia de la tasa de aprendizaje (α)

1.1.1. Escenario A (α_{slow}): convergencia lenta

Valor elegido: α = 1e-05

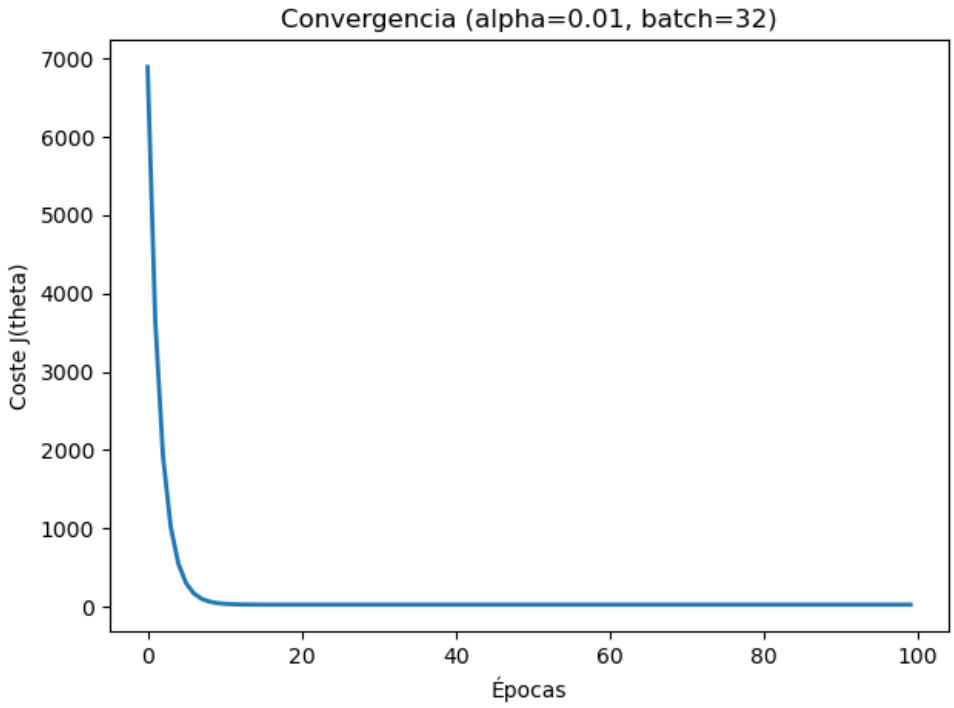


Análisis: ¿Por qué baja tan lentamente? ¿Cuántas iteraciones estimas para llegar cerca del mínimo?

Baja tan lentamente porque el tamaño de cada paso de actualización es minúsculo (el gradiente del error se multiplica por un α muy pequeño), por lo que los pesos apenas se modifican en cada iteración. Proyectando esta suave pendiente inicial, se estima que harían falta decenas de miles de iteraciones para acercarse de forma significativa al mínimo global.

1.1.2. Escenario B (α_{opt}): "curva de codo"

Valor elegido: $\alpha = 0.01$

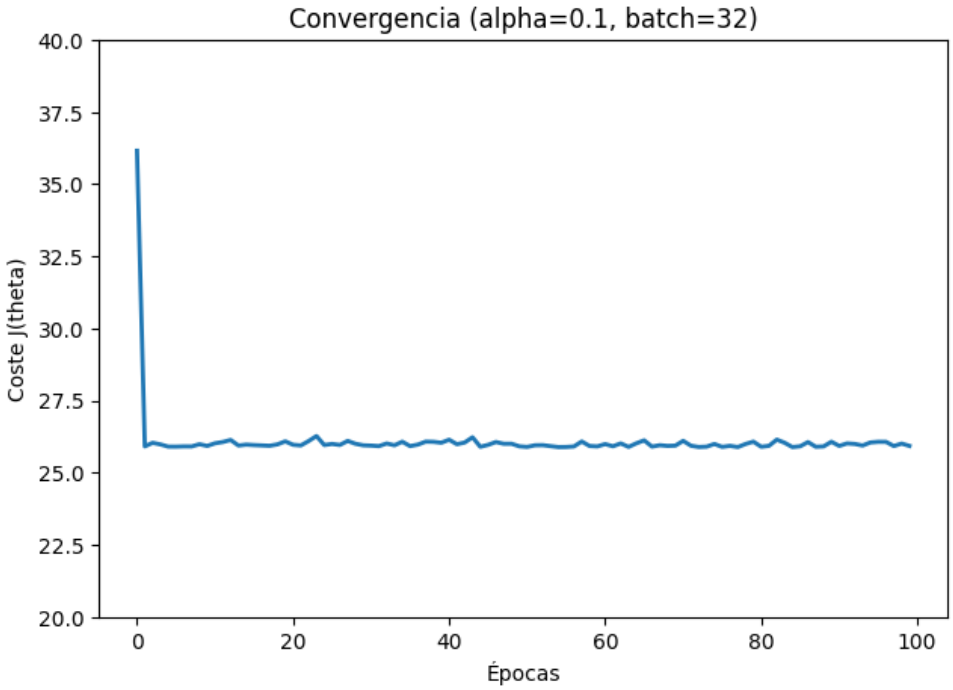


Análisis: Describe la "curva de codo" y qué te indica sobre rapidez y estabilidad.

La "curva de codo" muestra una caída inicial muy abrupta seguida de un aplanamiento drástico de la pendiente. Esto evidencia un comportamiento óptimo: el algoritmo avanza muy rápido cuando está lejos del mínimo y, al llegar al fondo del valle, los pasos del gradiente se ajustan milimétricamente sin rebotar, logrando así una gran rapidez en la convergencia y una estabilidad absoluta.

1.1.3. Escenario C (α_{osc}): oscilación amortiguada

Valor elegido: $\alpha = 0.1$

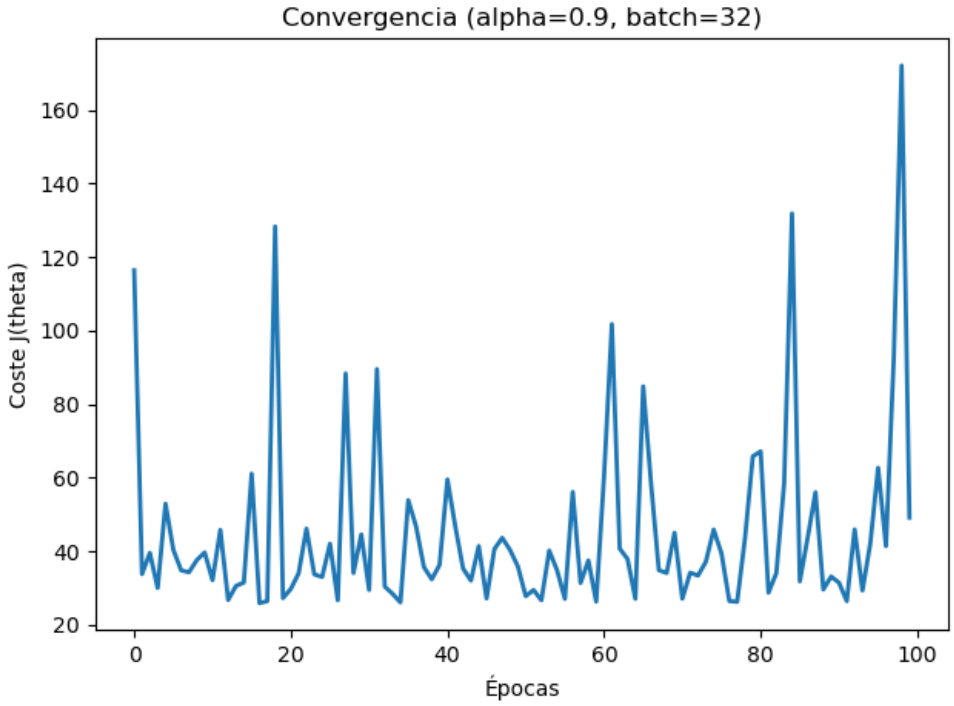


Análisis: ¿Qué ocurre con θ en el espacio de búsqueda? ¿Por qué oscila y puede estabilizarse?

En el espacio de búsqueda paramétrica, los pesos θ están sufriendo desplazamientos de magnitud excesiva y "saltan" repetidamente de una ladera a la opuesta del valle de coste, sobrepasando temporalmente el mínimo. Se observan oscilaciones por estos constantes rebotes; no obstante, el sistema logra estabilizarse porque la magnitud del verdadero gradiente disminuye gradualmente, atenuando la amplitud de los saltos hasta converger en la solución.

1.1.4. Escenario D (α_{fail}): divergencia

Valor elegido: $\alpha = 0.9$



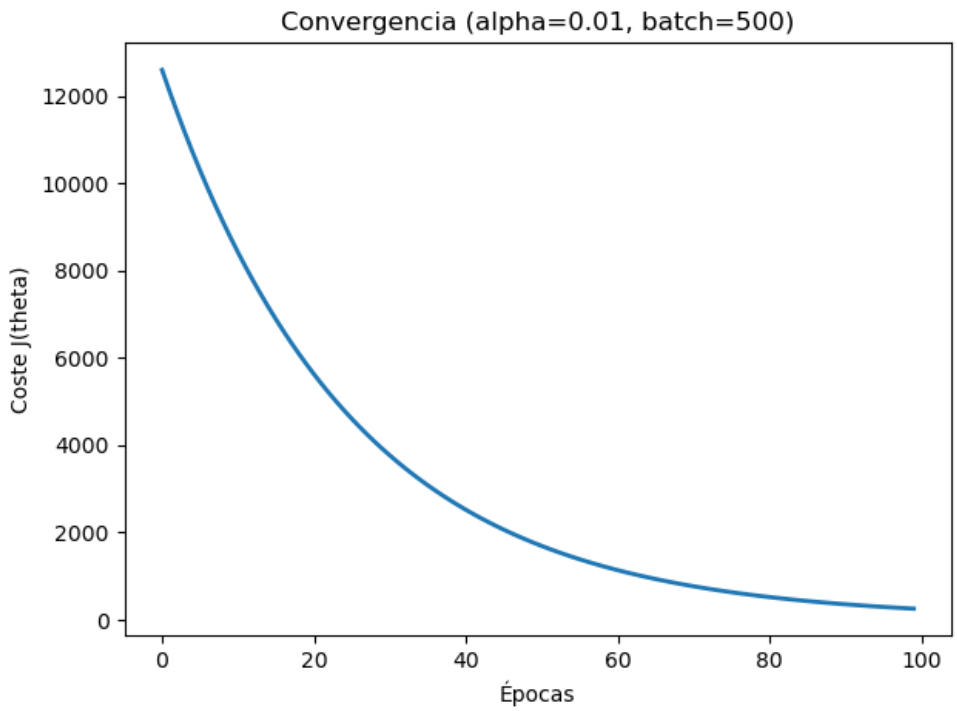
Análisis: ¿Por qué el error crece? Relaciónalo con "pasarse" del mínimo.

La función de coste diverge de forma monótona ascendente porque el coeficiente α es tan desproporcionado que el algoritmo "se pasa de largo" del mínimo local con una inercia destructiva. En lugar de descender gravitando hacia el valle, la gran magnitud de la corrección empuja a los pesos a rebotar cada vez a mayor altitud en las paredes opuestas, alejando indefinidamente la solución.

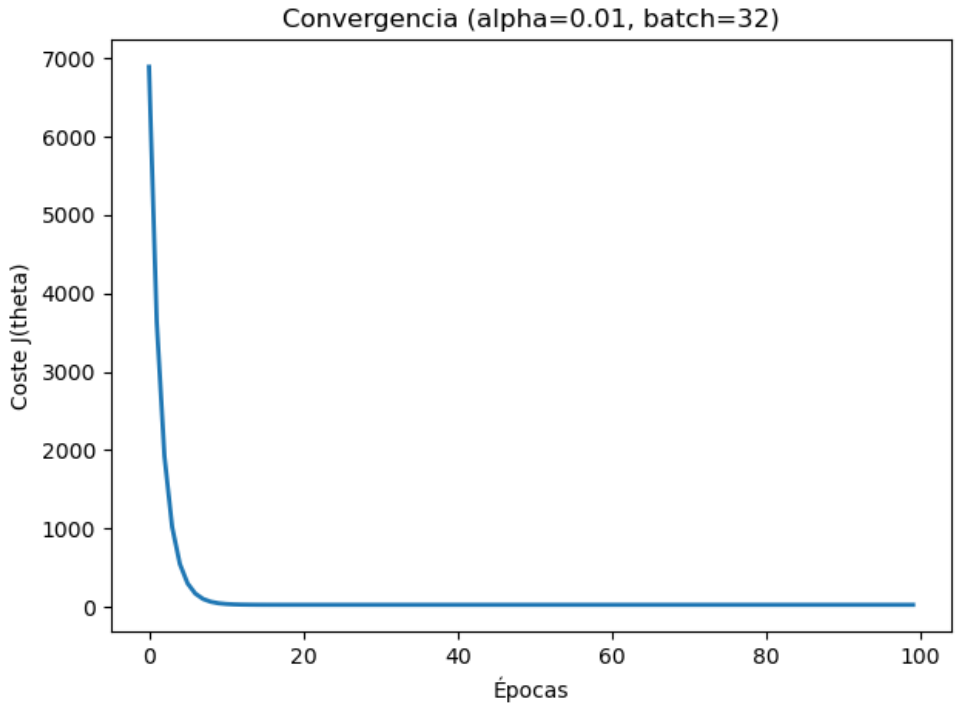
1.2. Tarea 2 — Compromiso del mini-batch

Usa tu α_{opt} y compara los siguientes esquemas de procesamiento:

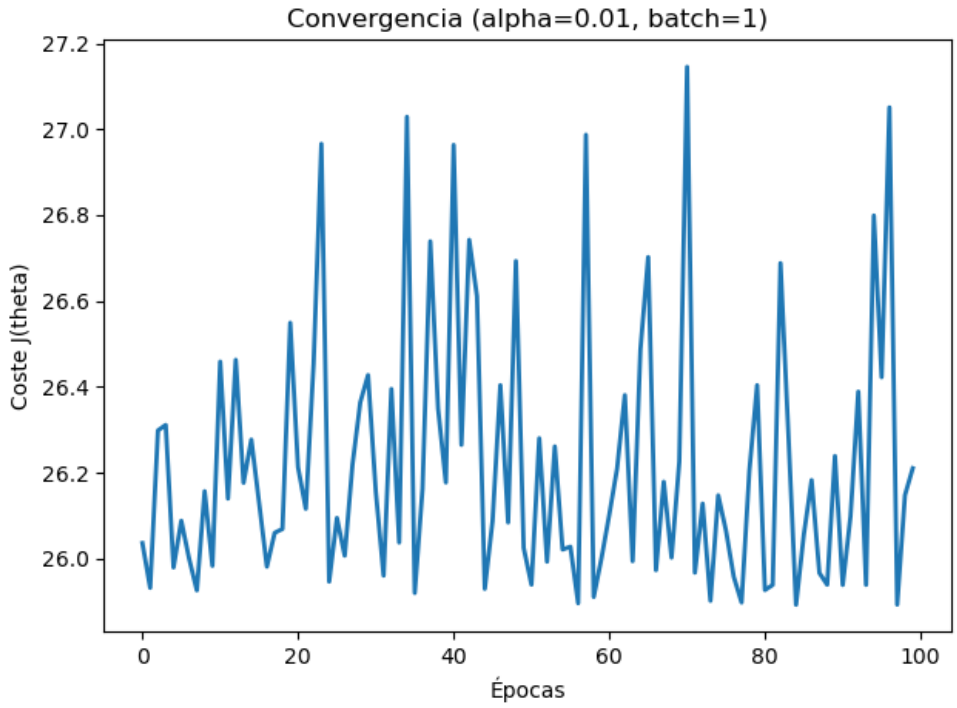
1.2.1. Batch completo



1.2.2. Mini-batch (32 o 16)



1.2.3. Estocástico puro (batch = 1)



Preguntas de reflexión:

1. ¿Cuál de las tres curvas es más "ruidosa" y por qué?

   La curva del gradiente estocástico puro (batch = 1) presenta el mayor nivel de ruido intrínseco. Fundamentar la dirección de máximo descenso en base a una única muestra observacional en cada iteración induce una elevadísima varianza estocástica. Esto provoca que el vector de descenso temporal apunte de forma muy errática, generando un fenotipo visual de dentado continuo (picos de subida y bajada). A pesar de esta inestabilidad local, la tendencia estocástica agregada garantiza un descenso a largo plazo.

2. A nivel de tiempo de ejecución (CPU), ¿cuál ha sido más eficiente? Justifica basándote en computación vectorial.

   A nivel hardware, tanto el Batch completo como el Mini-batch son radicalmente más eficientes que el descenso estocástico a base de iteraciones unitarias. Esto radica en el paradigma de la computación vectorial y las arquitecturas de instrucciones SIMD (Single Instruction, Multiple Data) del procesador. Agrupar los datos en bloques matriciales sustanciales permite a librerías numéricas de bajo nivel procesar transformaciones matemáticas complejas en estricto

paralelo y directamente en la ALU. Por el contrario, el modelo estocástico está condicionado a ejecutar un costoso control de flujo secuencial (bucles for ineficientes), provocando un cuello de botella abrumador en el tiempo de ejecución. El modelo **Mini-batch** resalta fundamentalmente por constituir un óptimo de Pareto: regulariza parcialmente el ruido de las muestras individuales garantizando un descenso suave, e invierte la carga computacional en veloces operaciones matriciales preservando simultáneamente una convergencia excepcionalmente rápida frente al Batch completo.

1.2.4. Tiempo de ejecución — Estocástico puro (batch = 1)

```
real    0m4.925s
user    0m4.886s
sys     0m0.039s
```

Figura 1.1: Tiempo de ejecución con batch=1.

1.2.5. Tiempo de ejecución — Mini-batch (batch = 32)

```
real    0m0.602s
user    0m0.568s
sys     0m0.033s
```

Figura 1.2: Tiempo de ejecución con batch=32.

1.2.6. Tiempo de ejecución — Batch completo (batch = 500)

```
real    0m0.495s
user    0m0.454s
sys     0m0.041s
```

Figura 1.3: Tiempo de ejecución con batch=500.

1.3. El reto — "Ajuste de precisión" (criterio de parada)

Implementa empíricamente el criterio de tolerancia algorítmica:

$$|J_t - J_{t-1}| < 10^{-5}$$

Para analizar las trayectorias asintóticas del sistema a largo plazo, se estableció un hiperparámetro deliberado que fijaba el límite en 2000 épocas en total. No obstante, al condicionar su diseño algorítmico al criterio de parada $|J_t - J_{t-1}| < 10^{-5}$ y equipar el modelo con su topología inicial más favorable ($\alpha = 0{,}01$, batch= 32), el algoritmo logra su punto de silla limitante ideal con una prontitud destacable y neutraliza la ejecución sistemáticamente en la **época 55**, obviando todo cómputo excedente e inútil que le sucediese al modelo a posterior.

```
real    0m0.435s
user    0m0.401s
sys     0m0.034s
```

Figura 1.4: Tiempo de convergencia interrumpida en la época 55 sujeto a $|J_t - J_{t-1}| < 10^{-5}$ para $\alpha = 0{,}01$ y batch= 32.

1.4. Conclusiones finales

El presente cuaderno de laboratorio corrobora categóricamente que la estabilidad e integridad de la convergencia global de un estimador iterativo requiere una rigurosa

sintonización analítica de sus hiperparámetros numéricos. Un factor paramétrico α marginal asegura la convergencia matemática, pero lo hace a expensas de un elevado lastre temporal infranqueable, mientras su hipertrofia causa la irremediable divergencia iterativa por retroproyección fuera del sumidero global. Por ello, el α óptimo impone un compromiso técnico de descenso inicial abrupto y mitigación suave sobre el vecindario absoluto. Respectivamente, se demuestra el segundo compromiso arquitectural: procesar muestras estocásticas individuales padece de extrema volatilidad métrica (ruido de las submuestras) y colapsa el motor computacional; el procesamiento de tipo Mini-batch emerge como el paradigma por excelencia superior, permitiendo actualizaciones continuas del vector de error al aprovechar el paralelismo subyacente de la microarquitectura del procesador de tipo vector (SIMD). Adicionalmente, el diseño del algoritmo ha reflejado categóricamente cómo implementar un control de flujo derivativo limitativo ($|J_t - J_{t-1}| < 10^{-5}$) extingue iteraciones marginales una vez garantizada la estabilización, culminando con un ahorro sustancial en el cálculo para el entrenamiento neuronal moderno.

Cuaderno de Laboratorio — Práctica 2: Selección de Modelos y Regularización

Objetivo

Dominar el protocolo de **Validación Cruzada (K-Fold)** y comprender cómo la regularización **Ridge (L2)** y **Lasso (L1)** previenen el sobreajuste y ayudan a la **selección de atributos**.

2.1. Tarea 1 — Estabilidad de la Validación Cruzada (K-Fold)

Configuración común

Modelo usado: **Regresión Lineal (Ridge con $\lambda \approx 0$)** (regresión lineal sin regularización)
Métrica: **RMSE** (RMSE)
Semillas: **`random_state` 1, 2, 3, 4 y 5, de forma consecutiva para la tarea 1, y el valor 42 para el resto.**

Hipótesis inicial (antes de ejecutar): ¿Crees que el RMSE será estable al cambiar K? No. Con valores muy bajos de K (ej. K=2), el error dependerá en gran medida de qué datos caigan aleatoriamente en entrenamiento y cuáles en test, provocando alta varianza.

2.1.1. Experimento A — K = 2 (Validación simple): variabilidad entre ejecuciones

Ejecuta el modelo **varias veces** con K=2 (por ejemplo 5–10) y registra el RMSE.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.33 | Time=0.0046s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.43 | Time=0.0050s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.24 | Time=0.0055s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.17 | Time=0.0046s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=2 | RMSE_CV=5.54 | Time=0.0057s
```

Registro de resultados (rellena):

Ejecución	RMSE
1	5.33
2	5.43
3	5.24
4	5.17
5	5.54

Análisis:

- ¿Por qué el error varía tanto entre ejecuciones? Porque el conjunto de entrenamiento es pequeño (50 % de los datos). El modelo es extremadamente sensible a la partición aleatoria específica de cada ejecución.
- ¿Es fiable esta métrica (con K=2) para tomar una decisión técnica? No, su alta varianza impide saber si una mejora aparente se debe a la configuración del modelo o simplemente a la aleatoriedad de la división de datos.

2.1.2. Experimento B — K = 10 (estándar): RMSE más estable

Ejecuta el modelo con K=10 y anota el RMSE.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.21 | Time=0.0190s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.19 | Time=0.0135s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.16 | Time=0.0154s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.17 | Time=0.0179s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=10 | RMSE_CV=5.23 | Time=0.0161s
```

RMSE observado (K=10): **5.19 (Media aprox.)**

Análisis: Describe si el resultado parece más estable que en K=2 y por qué. Sí, es notablemente más estable (fluctúa entre 5.16 y 5.23). Al usar K=10, el modelo entrena con el 90 % de los datos en cada iteración y promedia 10 errores diferentes, mitigando drásticamente el impacto de particiones anómalas.

2.1.3. Experimento C — K = 100 (Leave-One-Out): coste vs. precisión

Ejecuta el modelo con K=100 (LOOCV) y observa el RMSE y el tiempo.

```
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1252s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1395s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.17 | Time=0.1390s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.18 | Time=0.1290s
❯ /home/jesus/miniconda3/envs/iaa/bin/python /home/jesus/IAA/P2_regula
rizacion/python/01_regularizacion_cv.py
Method=LASSO | lambda=1.0 | K=100 | RMSE_CV=5.19 | Time=0.1231s
```

RMSE observado (K=100): **5.18** Tiempo aproximado: **0.131s**

Análisis: Compara el coste computacional frente a K=10. El tiempo de ejecución se multiplica casi por 8 (de $\sim$ 0,016s a $\sim$ 0,131s). ¿Merece la pena el tiempo extra para mejorar la precisión obtenida? No merece la pena. La mejora en la precisión del RMSE es marginal (apenas 0.01 unidades), lo cual no justifica el enorme aumento del coste computacional.

2.1.4. Resumen comparativo (K = 2 vs 10 vs 100)

K	RMSE	Comentario de estabilidad/coste
2	5.17 - 5.54	Alta inestabilidad (varianza). Coste muy bajo.
10	5.16 - 5.23	Alta estabilidad. Coste equilibrado y eficiente.
100	5.17 - 5.19	Máxima estabilidad. Coste computacional prohibitivo.

Conclusión Tarea 1: ¿Cuál elegirías para este problema y por qué? Elegiría K=10. Demuestra ser el estándar óptimo porque ofrece un error robusto y fiable para comparar modelos, sin incurrir en los altos tiempos de ejecución que exige K=100. Proporciona el mejor compromiso técnico entre precisión estadística y eficiencia temporal.

2.2. Tarea 2 — Regularización Ridge vs. Lasso (K = 10)

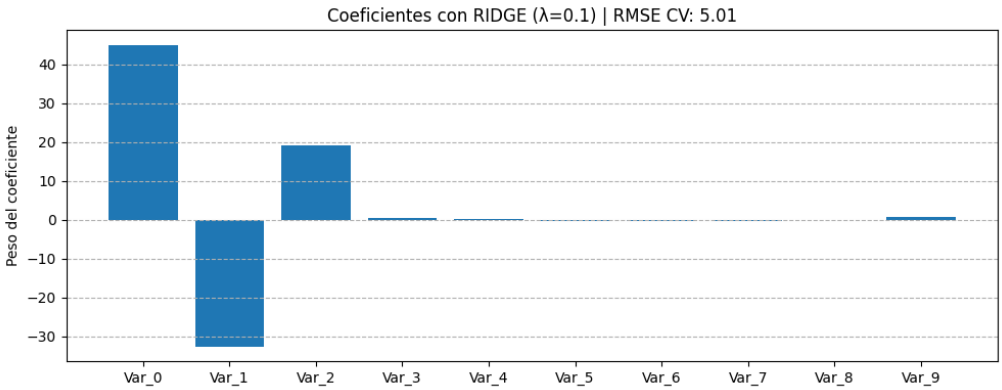
Configuración

Fijamos K = 10 y comparamos:

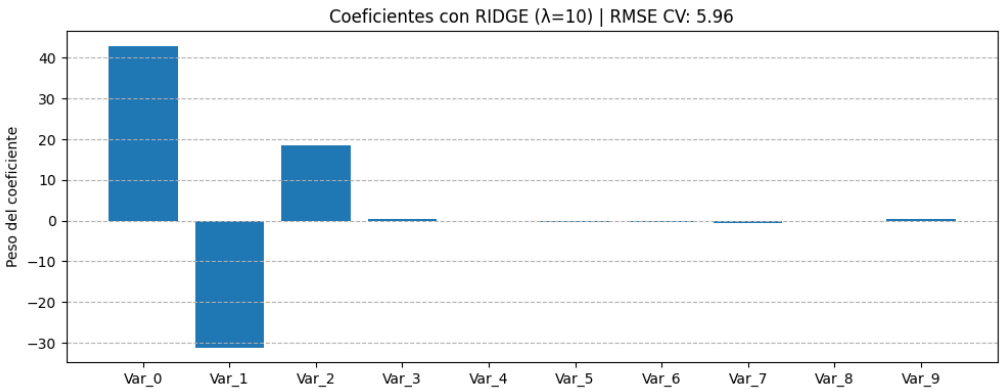
- **Ridge Regression** (penalización L2)
- **Lasso Regression** (penalización L1)

2.2.1. Ridge (L2) — $\lambda = 0{,}1,\ 10,\ 100$

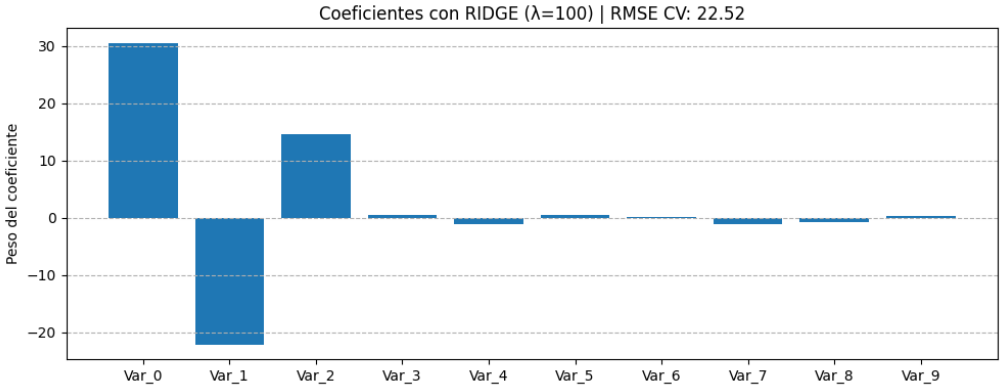
Resultados: captura el gráfico de barras de coeficientes para cada λ.



RMSE Ridge ($\lambda = 0{,}1$): **5.01**



RMSE Ridge ($\lambda = 10$): **5.96**

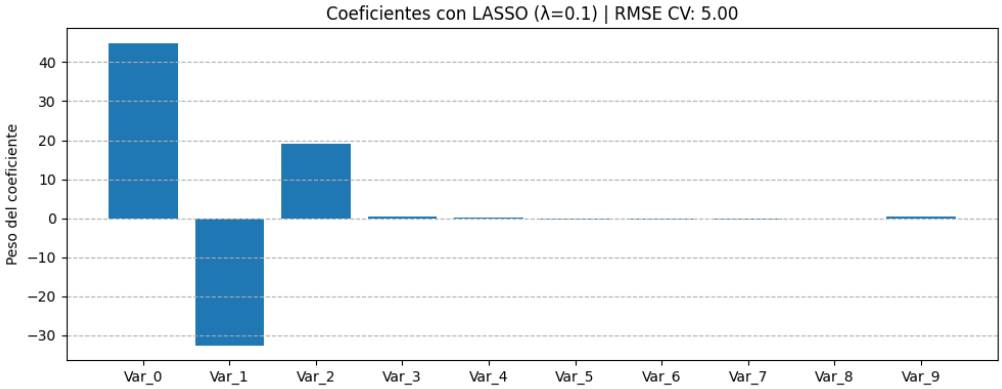


RMSE Ridge ($\lambda = 100$): **22.52**

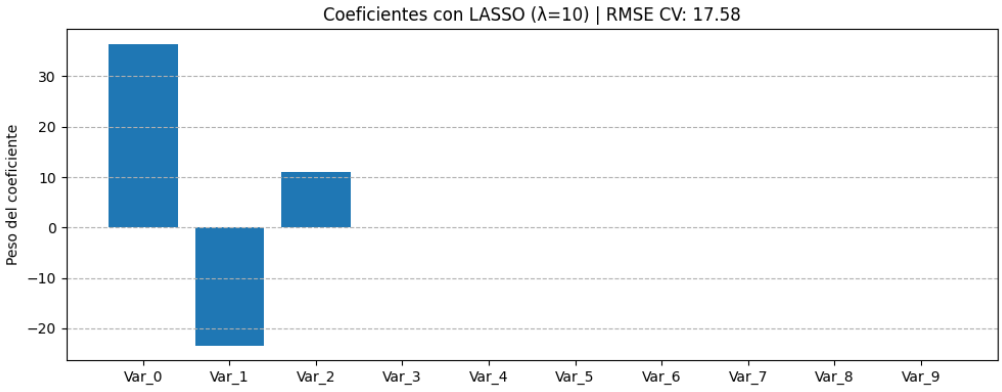
Análisis: ¿Llegan a valer **cero** los coeficientes de las variables ruidosas?. No llegan a valer cero absoluto. Visualmente en las gráficas parece que algunas como (`Var_8` desaparecen, pero técnicamente Ridge (L2) simplemente reduce o .^en^coge"drásticamente la magnitud de los coeficientes penalizados, distribuyendo el peso entre todos sin llegar a anularlos por completo.

2.2.2. Lasso (L1) — $\lambda = 0{,}1,\ 10,\ 100$

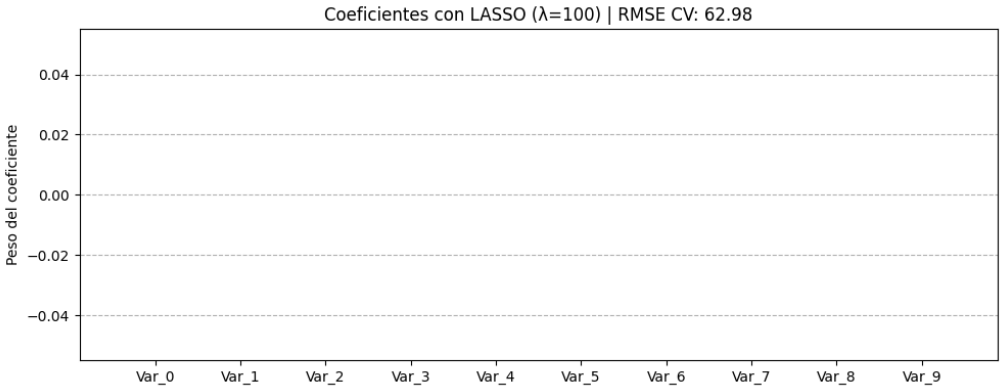
Resultados: captura el gráfico de barras de coeficientes para cada λ.



RMSE Lasso ($\lambda = 0{,}1$): **5.00**



RMSE Lasso ($\lambda = 10$): **17.58**



RMSE Lasso ($\lambda = 100$): **62.98**

Análisis clave:

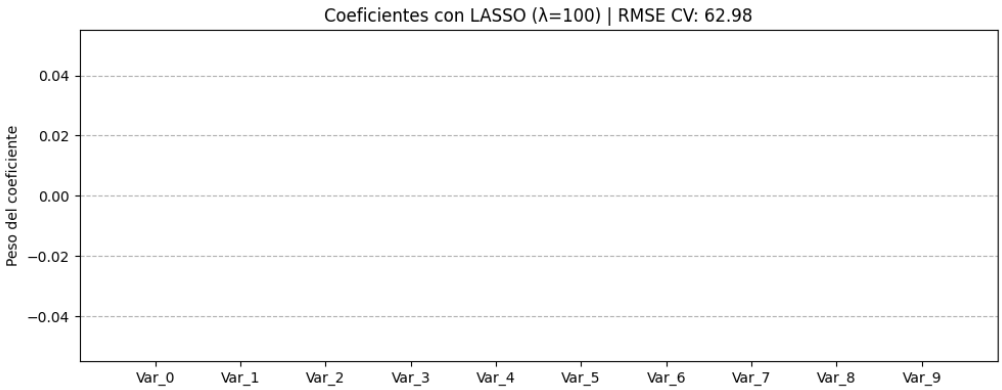
1. Identifica para qué valor de λ el modelo **Lasso** consigue **eliminar** (poner a cero) las 7 variables ruidosas, quedándose solo con las 3 importantes:

$$\lambda^* = \mathbf{1.0}(\text{y en general entre 0.5 y 2.0})$$

2. Variables con coeficiente distinto de cero (modelo Lasso con λ^*):
Var_0, Var_1, Var_2

2.2.3. El dilema de la complejidad — λ demasiado alto

Prueba un valor extremo, por ejemplo $\lambda = 100$, y registra qué ocurre.



RMSE ($\lambda = 100$): **62.98**

Análisis: Explica qué ocurre con el RMSE si aplicas un λ demasiado alto. El RMSE se dispara exageradamente (hasta 62.98). ¿Estamos ante un caso de **sesgo alto** o **varianza alta**? Justifica. Estamos ante un caso evidente de **sesgo alto (underfitting)**. La penalización es tan severa que el modelo se vuelve excesivamente rígido, empujando todos los coeficientes hacia cero e impidiendo que pueda capturar la señal o patrón real de los datos.

2.3. Reto — El “Cazador de Variables”

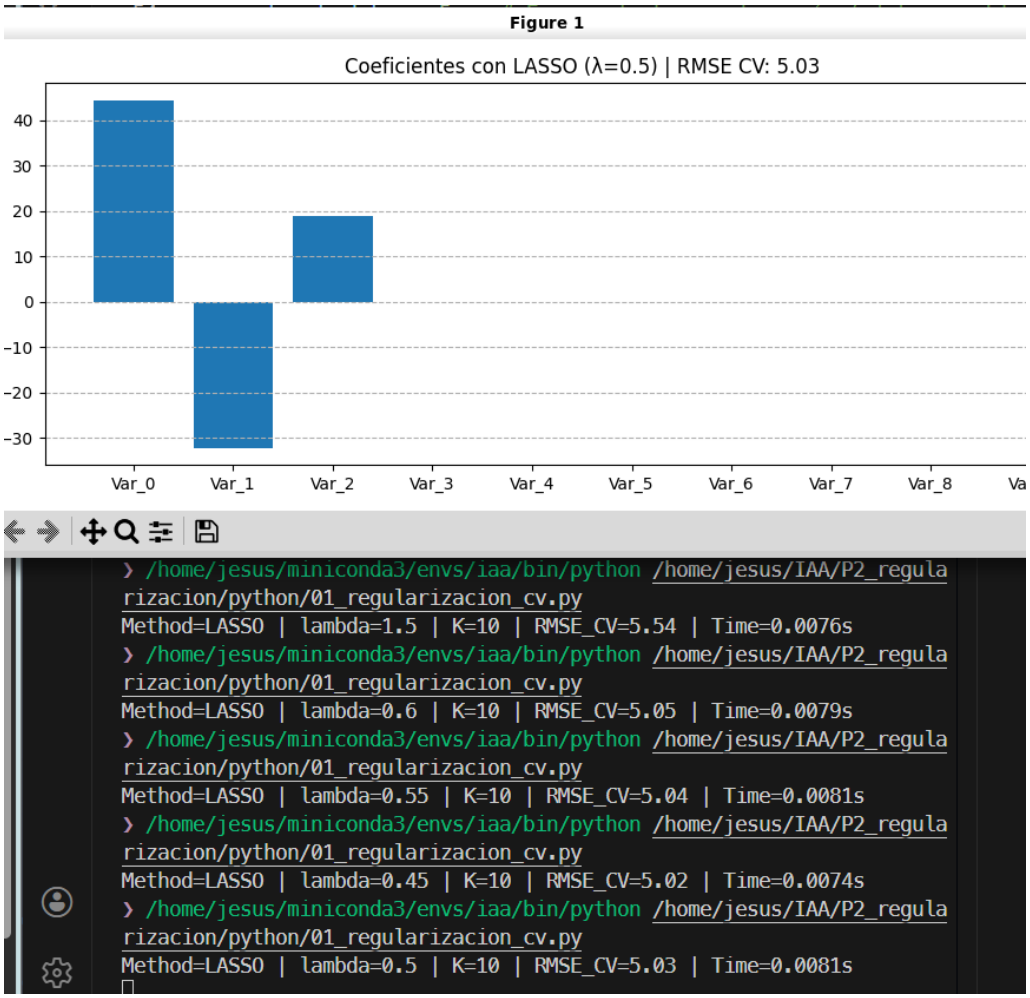
Contexto: Imagina que cada variable incluida en tu modelo tiene un **coste de mantenimiento**. Tu objetivo es encontrar el modelo **más sencillo posible** (menos variables con coeficientes $\neq 0$) que mantenga un RMSE inferior a un umbral:

$$\text{RMSE} < \mathbf{5.50} \quad \text{(umbral asumido)}$$

Mejor solución encontrada (rellena):

$$\lambda = \mathbf{0.5} \qquad \#\text{Variables activas} = \mathbf{3} \qquad \text{RMSE} = \mathbf{5.03}$$

Variables seleccionadas (coeficiente $\neq 0$): **Var_0, Var_1 y Var_2**



Justificación técnica (6–10 líneas): ¿Por qué este modelo es el mejor compromiso entre simplicidad (coste) y rendimiento (RMSE)? Tras realizar un ajuste paramétrico fino, $\lambda = 0{,}5$ demuestra ser el punto de equilibrio óptimo. Logra la máxima simplicidad técnica al descartar completamente los 7 atributos que solo aportaban ruido, minimizando así los futuros costes de mantenimiento e ingesta de datos del modelo. Simultáneamente, conserva un rendimiento excelente con un RMSE de 5.03, el cual es prácticamente idéntico al de modelos menos restrictivos (como $\lambda = 0{,}1$ con RMSE 5.00). Nos quedamos con la señal real y descartamos el ruido sin penalizar el error.

2.4. Conclusiones finales

Resume en 8–12 líneas lo aprendido sobre:

- Estabilidad de K-Fold al variar K (K=2 vs K=10 vs LOOCV).
- Diferencias prácticas entre Ridge y Lasso (coeficientes, selección de variables).
- Relación entre λ grande y el sesgo/varianza.

La validación cruzada con $K = 10$ es el estándar porque equilibra un RMSE estadísticamente fiable frente a la inestabilidad de $K = 2$, evitando los desproporcionados tiempos de ejecución de LOOCV ($K = 100$). En cuanto a la regularización, Ridge (L2) reduce la magnitud de los coeficientes de las variables menos relevantes pero los mantiene activos, mientras que Lasso (L1) es capaz de forzarlos a cero absoluto. Esta propiedad hace que Lasso actúe como un potente selector automático de características, ideal para limpiar datasets con ruido. Por último, un hiperparámetro λ demasiado alto penaliza en exceso al modelo, volviéndolo incapaz de aprender (underfitting) y empujándolo a una zona de sesgo alto donde el error se dispara.

Cuaderno de Laboratorio — Práctica 3: Metodología de Evaluación y Rigor Científico

Objetivo

Aprender a realizar evaluaciones honestas en condiciones difíciles, especialmente con datos desbalanceados, y detectar errores metodológicos graves como la fuga de datos (*data leakage*).

Material de partida

Se parte de:

- Un dataset sintético de **1000 pacientes** con **5 variables predictoras**.
- Una clase positiva muy minoritaria (**2 %** de los casos), es decir, $K = 20$ de $N = 1000$.

3.1. Tarea 1 — La lotería de la partición aleatoria

Configuración común

Modelo usado: `LogisticRegression` (con `class_weight='balanced'`)
Métricas usadas: `Nº positivos en test`, `% clase 1`
Semilla(s): `0, 1, 2, 3, 4` (5 ejecuciones aleatorias)

Hipótesis inicial: Es probable que el número de positivos en test sea muy inestable. Al haber solo un 2 % de casos positivos (20 de 1000), un split aleatorio del 20 % (200 ejemplos) podría coger muy pocos positivos por puro azar, o incluso ninguno. Esto dejaría el test inservible.

3.1.1. Análisis estadístico de la partición

Para comprobar el riesgo real de hacer una mala partición, he calculado la probabilidad usando la distribución hipergeométrica. Teniendo $N = 1000$ pacientes, $K = 20$ positivos en total, y sacando una muestra para test de $n = 200$:

Lo normal (la media esperada) es que caigan 4 positivos en test:

$$\mathbb{E}[X] = \frac{200 \cdot 20}{1000} = 4$$

Pero, ¿qué pasa si tenemos mala suerte? Calculando la probabilidad de que no caiga **ningún** positivo ($X = 0$):

$$P(X = 0) = \frac{\binom{20}{0}\binom{980}{200}}{\binom{1000}{200}} \approx 0{,}0110 \quad (1{,}10\,\%)$$

Si sumamos la probabilidad de sacar 0 o 1 positivo, el riesgo sube a casi un 7 %:

$$P(X \leq 1) \approx 0{,}0110 + 0{,}0563 = 0{,}0673 \quad (6{,}73\,\%)$$

¿Por qué es un problema? Si en el test no hay positivos ($X = 0$), las métricas se rompen:

- **Recall**: No se puede calcular (0/0) porque no hay verdaderos positivos ni falsos negativos reales.
- **F1-score**: Al depender del Recall, también da error.
- **AUC-ROC**: La librería devuelve `NaN` porque necesita ejemplos de ambas clases para pintar la curva.

Conclusión: un mal split aleatorio puede hacer que la evaluación no sirva para nada, sin importar qué modelo usemos.

3.1.2. Registro de resultados (5 particiones aleatorias)

Ejecución	Positivos en test	% clase 1 en test
1	4	2,0 %
2	3	1,5 %
3	5	2,5 %
4	4	2,0 %
5	3	1,5 %

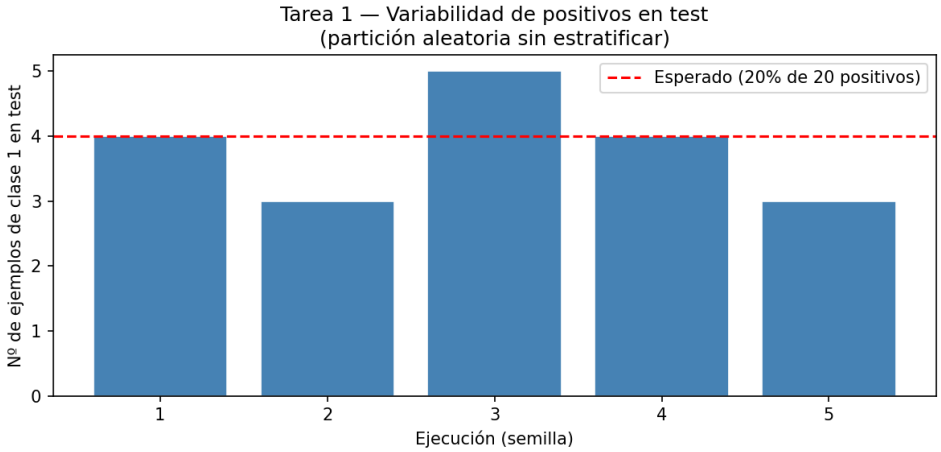


Figura 3.1: Variabilidad del número de ejemplos positivos en el conjunto de test según la semilla. La línea roja indica el valor esperado teórico (4).

Análisis de los resultados:

- *¿Ha cambiado mucho el número de positivos en test entre ejecuciones?* Sí, el número oscila entre 3 y 5 en mis ejecuciones. Aunque numéricamente parezca poco, en un test tan pequeño supone una variación del $\pm 25\,\%$, lo que altera bastante las métricas.

- *¿Sería fiable la evaluación si el azar dejara solo 0 o 1 positivos en test?* Para nada. Como he demostrado antes, hay un $\approx 6{,}73\,\%$ de probabilidad de que ocurra. Con 0 positivos las métricas dan error matemático, y con 1 solo positivo el resultado dependería al $100\,\%$ de acertar o fallar en un único paciente, perdiendo todo el rigor estadístico.

3.2. Tarea 2 — Partición estratificada

3.2.1. Registro por folds (`StratifiedKFold`, $k = 10$)

Fold	Positivos en test	Tamaño del fold	Proporción clase 1
1	2	100	2,0 %
2	2	100	2,0 %
3	2	100	2,0 %
4	2	100	2,0 %
5	2	100	2,0 %
6	2	100	2,0 %
7	2	100	2,0 %
8	2	100	2,0 %
9	2	100	2,0 %
10	2	100	2,0 %

Desviación típica de la proporción de clase 1 entre folds: **0,0000 %** (prácticamente nula, lo que confirma que la estratificación funciona correctamente).

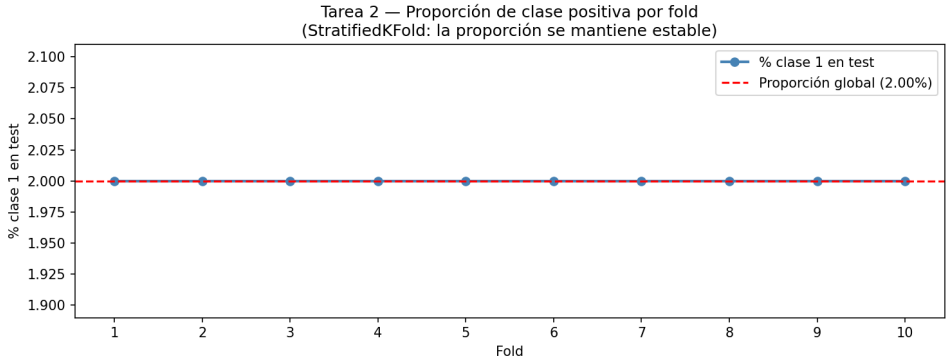


Figura 3.2: Proporción de la clase positiva por fold con `StratifiedKFold`. La línea roja indica la proporción global del dataset (2 %).

Análisis: `StratifiedKFold` garantiza que cada fold mantenga la misma proporción de clases que el dataset original (2 %). En datasets muy desbalanceados, esto evita que algunos folds se queden sin muestras de la clase minoritaria, asegurando que el modelo sea evaluado siempre sobre una distribución representativa y reduciendo la varianza debida al azar.

3.3. Tarea 3 — Comparativa de la varianza

3.3.1. Resumen de resultados (10 iteraciones, 10 folds)

La tabla 3.1 recoge la media y la desviación típica de las tres métricas calculadas mediante `cross_val_score` durante 10 iteraciones independientes de validación cruzada con 10 *folds*. He añadido la métrica AUC-ROC para reforzar las conclusiones sobre la necesidad de estratificación.

Tabla 3.1: Comparativa de estabilidad de métricas: KFold estándar vs. StratifiedKFold (10 iteraciones × 10 folds).

Método	**Accuracy**		**F1-macro**		**AUC-ROC**	
	Media	*std*	Media	*std*	Media	*std*
`KFold`	0,5771	0,0090	0,3829	0,0051	**N/D** (*8/10 iters.*†)	
`StratifiedKFold`	0,5783	**0,0085**	0,3857	**0,0050**	**0,5283**	**0,0253**

† **N/D = No Definida.** En 8 de las 10 iteraciones de `KFold`, algún fold del conjunto de test careció de ejemplos positivos. En esas condiciones, `roc_auc_score` devuelve `NaN` (indefinido), puesto que la curva ROC no puede construirse sin ambas clases representadas. Solo en las 2 iteraciones en que por azar todos los folds tuvieron al menos un positivo se obtuvo un valor numérico (AUC ≈ 0,49). Este resultado es la prueba empírica más directa de la necesidad de estratificación: sin ella, la métrica de evaluación más robusta para clases desbalanceadas queda matemáticamente indefinida en la mayoría de los experimentos.

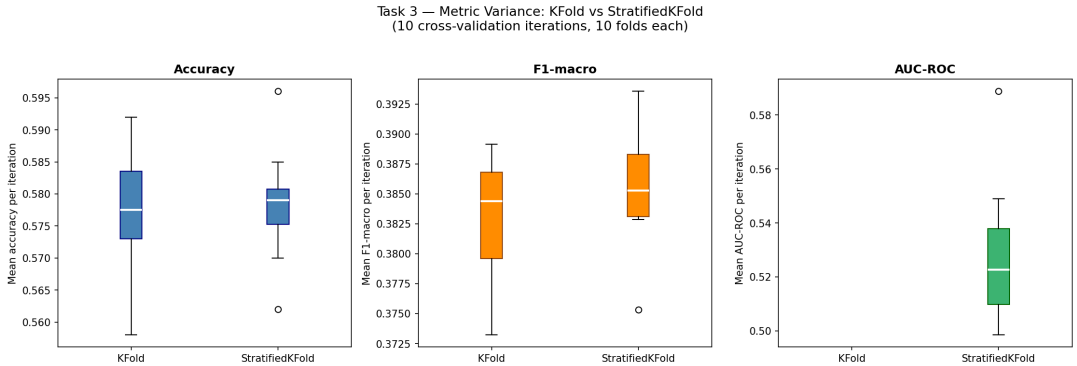


Figura 3.3: Distribución de Accuracy, F1-macro y AUC-ROC a lo largo de 10 iteraciones de validación cruzada (10 *folds*). Cada métrica se muestra como un diagrama de cajas (*boxplot*). `StratifiedKFold` produce cajas más estrechas en las tres métricas, evidenciando menor varianza y mayor estabilidad en la evaluación.

Interpretación:

- *¿Qué método produce métricas más estables?* El método `StratifiedKFold` produce métricas más estables, como se observa en la menor desviación típica (*std*) en Accuracy y F1-macro; pero la diferencia más llamativa se produce en **AUC-ROC**.
- *¿Dónde se nota más la mejora?* En AUC-ROC, sin duda. Con `KFold estándar`, la métrica es **indefinida (`NaN`)** en 8 de las 10 iteraciones, porque algunos folds quedan vacíos de ejemplos positivos. Solo por azar, en 2 iteraciones se obtiene un valor numérico ($\approx$ 0,49), que además es desfavorable (cercano a una clasificación aleatoria). Con `StratifiedKFold`, la AUC-ROC es siempre computable: media = 0,5283, desv. típica = 0,0253, en las 10 iteraciones.
- **La AUC-ROC reafirma empíricamente la necesidad de estratificar:** el hecho de que `KFold` haga la métrica indefinida en el 80 % de los experimentos no es un artefacto numérico, sino la *manifestación directa* del problema demostrado matemáticamente en la Tarea 1 mediante la distribución hipergeométrica: cuando los folds no están estratificados, es probable que algunos queden sin representantes de la clase positiva, haciendo imposible cualquier evaluación orientada a esa clase. La AUC-ROC, al ser la métrica más exigente en ese sentido, lo pone de manifiesto de forma inequívoca.

3.4. Tarea 4 — Detección de *data leakage*

3.4.1. Comparativa: con y sin variable trampa

Escenario	Accuracy	F1-macro
Sin variable trampa (*evaluación real*)	0,5850	0,3887
Con variable trampa (`ID_Hospital_Filtro`)	**1,0000**	**1,0000**

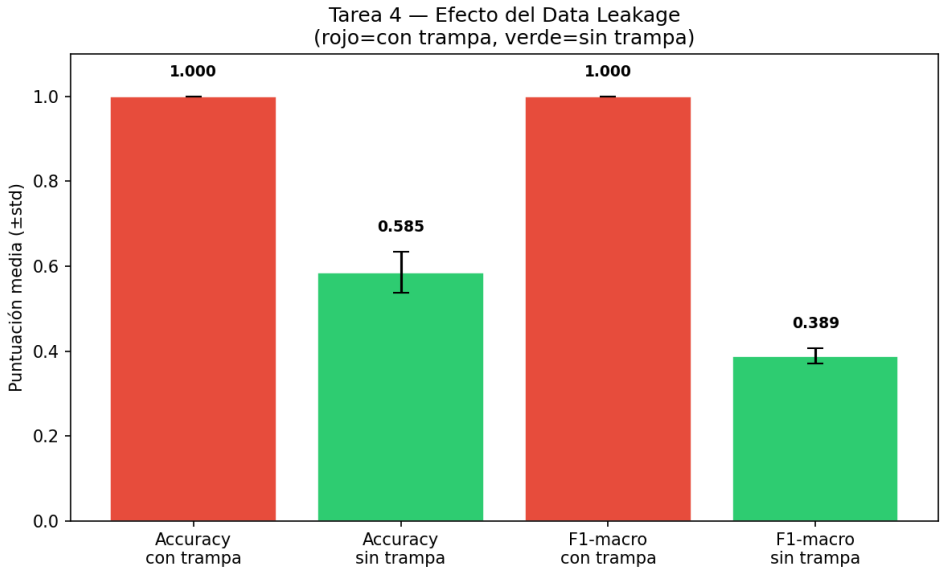


Figura 3.4: Efecto del *data leakage*. Las barras rojas corresponden al escenario con la variable trampa; las verdes, al escenario real sin ella.

Preguntas:

- *¿Qué variable está filtrando información del objetivo?* La variable `ID_Hospital_Filtro`.
- *¿Por qué esa variable invalida la evaluación?* Ha sido construida con una correlación perfecta (o casi perfecta) con la clase objetivo. El modelo aprende a usar este "filtro" en lugar de las variables clínicas reales, obteniendo un rendimiento del 100 % que es imposible en la práctica real.
- *¿Diferencia entre un modelo "bueno" y uno contaminado por leakage?* A diferencia de un modelo legítimo, uno con leakage simplemente está haciendo trampa usando información que no estará disponible en producción, por lo que su 100 % de acierto es un espejismo.

3.5. Reto — Auditoría metodológica

1. **Partición aleatoria:** Como hemos comprobado, corremos el riesgo de dejar el test sin apenas casos positivos, lo que invalida cualquier métrica que intentemos calcular.

2. **Estratificación:** Es la solución directa al problema anterior, ya que nos garantiza una representación mínima de la clase minoritaria en cada fold para poder evaluar de forma justa.

3. **Fuga de datos (Leakage):** Invalida por completo el experimento. El modelo no aprende, solo usa la variable trampa como atajo, por lo que fallará estrepitosamente en el mundo real.

4. **Buenas prácticas:** Usar siempre estratificación, auditar variables en busca de correlaciones sospechosas, emplear métricas robustas (F1, AUC-ROC) y documentar la reproducibilidad mediante semillas y validación cruzada repetida.

3.6. Conclusión final

Esta práctica me ha dejado claro que evaluar un modelo no es solo ejecutar unas líneas de código y mirar la *Accuracy*: con datos desbalanceados, hay que tomar decisiones de diseño concretas o los números que obtenemos nos van a engañar.

En cuanto a la **partición**, calculé con la distribución hipergeométrica que hay un $P(X = 0) \approx 1{,}10\,\%$ de probabilidad de que el test quede sin ningún positivo, y casi un 7 % de terminar con uno solo. En esas condiciones, F1-score, Recall y AUC-ROC son directamente indefinidas. Eso ya es razón suficiente para usar siempre estratificación: no es opcional si queremos que los resultados tengan sentido.

Los resultados obtenidos con 10 iteraciones de validación cruzada lo confirman: con `KFold` estándar, la AUC-ROC fue `NaN` en 8 de las 10 iteraciones porque algunos folds se quedaron sin positivos; con `StratifiedKFold` siempre fue computable y más estable (*std* menor en las tres métricas).

En cuanto al **data leakage**, el caso de `ID_Hospital_Filtro` es bastante claro: una variable artificialmente correlacionada con el target hace que el modelo alcance un 100 % de Accuracy, lo que en la práctica significaría entregar un sistema que falla en cuanto se usa con datos reales. La regla práctica que saco de aquí es auditarlas siempre antes de entrenar.

Cuaderno de Laboratorio — Práctica 4: Clasificación y el Valor del Error (Matriz de Costes)

Objetivo

Entrenar un clasificador probabilístico para un problema de detección de fraude bancario y estudiar cómo el umbral de decisión afecta a los errores cometidos y al coste total. El objetivo central no es únicamente medir el rendimiento predictivo, sino comprender el impacto económico real de las decisiones del modelo a partir de una **matriz de costes asimétrica**.

Descripción del problema

Se trabaja con un problema de clasificación binaria sobre el dataset `fraude_transacciones.cs`

- **Clase 0**: transacción normal.
- **Clase 1**: transacción fraudulenta.

Los errores tienen costes muy distintos:

- **Falso Negativo (FN)**: no detectar un fraude → 200 €
- **Falso Positivo (FP)**: bloquear una transacción legítima → 10 €

La decisión final no debe evaluarse solo por el número de aciertos, sino por el **coste económico** asociado a los errores.

4.1. Tarea 1 — Análisis con el umbral por defecto ($\tau = 0{,}5$)

4.1.1. Modelo utilizado

Se ha entrenado una **Regresión Logística** (`LogisticRegression`, `max_iter=1000`) sobre el 70 % del dataset (partición estratificada, `random_state=42`) y evaluado sobre el 30 % restante.

4.1.2. Matriz de confusión

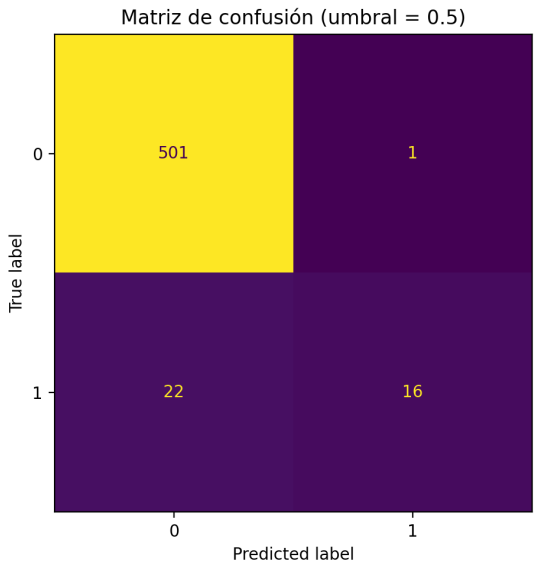


Figura 4.1: Matriz de confusión para el umbral estándar $\tau = 0{,}5$.

A partir de la matriz de confusión detallada en la Figura 4.1, se obtienen los siguientes valores:

- Verdaderos Positivos (TP): **16**
- Verdaderos Negativos (TN): **501**
- Falsos Positivos (FP): **1**
- Falsos Negativos (FN): **22**

4.1.3. Cálculo del coste total

$$\text{Coste Total} = (FP \cdot 10) + (FN \cdot 200) = (1 \cdot 10) + (22 \cdot 200) = \mathbf{4410}\,€$$

4.1.4. Interpretación

El umbral por defecto de 0.5 **no es razonable** en este contexto. Aunque el modelo comete muy pocos falsos positivos (solo 1), deja pasar 22 transacciones

fraudulentas. Dado que el coste de un falso negativo (200 €) es veinte veces el de un falso positivo (10 €), el impacto económico resulta muy elevado. En un sistema de detección de fraude es fundamental minimizar los errores más caros; usar este umbral sin revisarlo sería un error de diseño grave.

4.2. Tarea 2 — Estudio de probabilidades y umbrales

4.2.1. Análisis de distintos umbrales

Se han extraído las probabilidades de la clase fraude para cada observación y se han evaluado umbrales $\tau \in [0, 1]$ en pasos de 0.05. La Tabla 4.1 resume los resultados para los primeros tramos y el extremo superior.

Tabla 4.1: Resumen de FP, FN y Coste Total para distintos umbrales de decisión.

Umbral	FP	FN	Coste Total (€)
0.00	502	0	5020
0.05	112	9	**2920**
0.10	42	14	3220
0.15	22	15	3220
0.20	13	15	3130
0.25	9	16	3290
0.30	4	17	3440
0.40	1	20	4010
0.50	1	22	4410
1.00	0	38	7600

4.2.2. Observaciones

Existe un claro compromiso (*trade-off*) entre los dos tipos de error al variar τ:

- **Cuando el umbral baja** (hacia 0): se exige menos confianza para clasificar como fraude. Caen los Falsos Negativos (detectamos más fraudes), pero aumentan drásticamente los Falsos Positivos (bloqueamos más transacciones legítimas).
- **Cuando el umbral sube** (hacia 1): las alertas son mucho más selectivas. Los Falsos Positivos desaparecen, pero los Falsos Negativos se disparan (dejamos escapar la mayoría de los fraudes).

El umbral de decisión debe ajustarse según la penalización económica de cada error, en lugar de asumir un valor estático por defecto.

Desde una perspectiva de ingeniería, disponer de las probabilidades en crudo (por ejemplo, un 0.30 de probabilidad de fraude) nos otorga un control total sobre la lógica de negocio.

Como ingeniero, ante la asimetría de costes mencionada (donde perder 200 € por un fraude no detectado duele mucho más que los 10 € de un bloqueo erróneo), se puede decidir de forma proactiva: "No voy a esperar a que el modelo esté un 50 % seguro para actuar; voy a bloquear cualquier transacción que supere el 20 % de probabilidad". Al bajar el umbral a $\tau = 0{,}20$, esa transacción con un 0.30 de probabilidad ahora es clasificada correctamente como fraude, permitiendo su bloqueo a tiempo.

En definitiva, al extraer las probabilidades es el diseñador del sistema quien decide dónde situar la línea de corte para equilibrar las pérdidas económicas entre Falsos Positivos (10 €) y Falsos Negativos (200 €), optimizando el sistema para el impacto real en el negocio.

4.3. Tarea 3 — Optimización del coste

4.3.1. Gráfica de coste frente a umbral

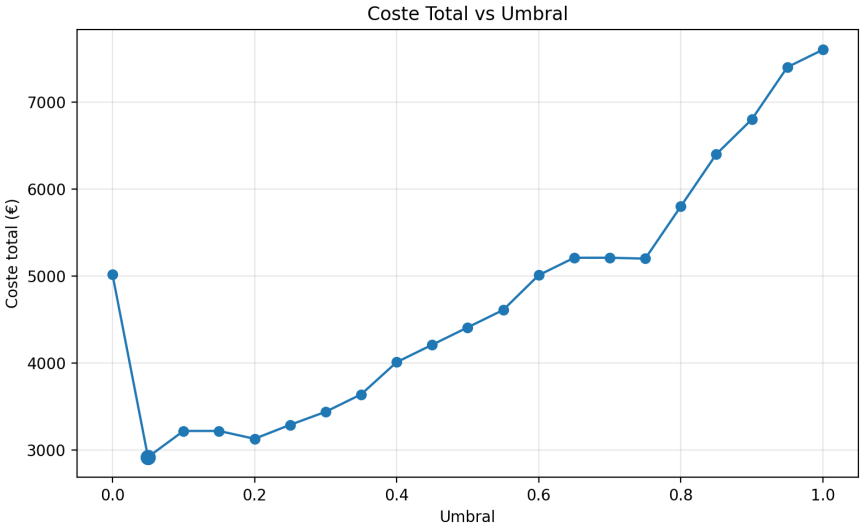


Figura 4.2: Coste total (€) en función del umbral de decisión. Como se aprecia en la Figura 4.2, el punto óptimo se alcanza en $\tau^* = 0{,}05$.

4.3.2. Umbral óptimo

En base a la gráfica anterior, identificamos el punto de menor coste:

$$\tau^* = 0{,}05 \qquad \text{Coste mínimo} = 2920\,€$$

4.3.3. Interpretación

El umbral óptimo ($\tau^* = 0{,}05$) es muy inferior a 0.5. Esto tiene todo el sentido dada la fuerte asimetría de la matriz de costes: el coste de un FN (200 €) es 20 veces el de un FP (10 €). Para minimizar el coste total conviene ser muy sensible a los fraudes, aunque eso suponga aceptar más falsos positivos. La estrategia más razonable para el banco es **conservadora**: es preferible molestar puntualmente a un cliente legítimo mediante un bloqueo temporal que permitir que un fraude se consume sin ser detectado.

4.4. Tarea 4 — Curvas ROC y Precision-Recall

4.4.1. Curva ROC

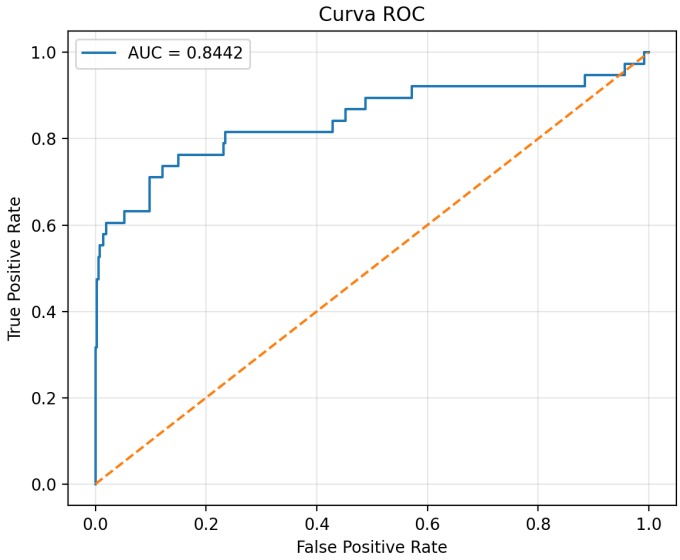


Figura 4.3: Curva ROC del clasificador. AUC-ROC = 0,8442.

4.4.2. Curva Precision-Recall

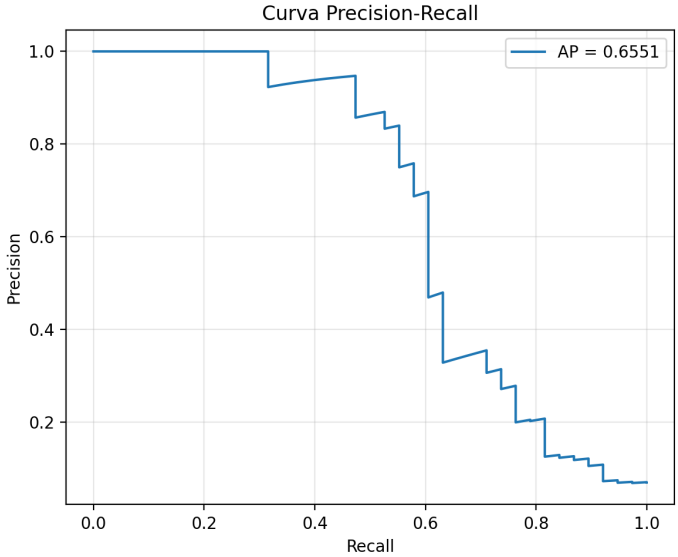


Figura 4.4: Curva Precision-Recall del clasificador. Average Precision = 0,6551.

4.4.3. Discusión

Al analizar un clasificador sobre un dataset desbalanceado, la curva Precision-Recall suele ser mucho más informativa que la ROC (Figura 4.3 frente a Figura 4.4). El motivo es estructural: el *False Positive Rate* de la curva ROC toma como denominador el total de negativos, que en este problema es muy numeroso. Como consecuencia, incrementos apreciables de falsos positivos apenas desplazan el FPR y el AUC-ROC se mantiene elevado (0.8442) pese a que el modelo comete errores relevantes sobre la clase fraude. La curva PR evita este efecto porque la Precisión (TP / (TP + FP)) cae en cuanto aumentan los FP, lo que la hace sensible exactamente a la calidad de recuperación de la clase minoritaria que interesa controlar.

4.5. Decisión final de ingeniería

A la vista de los resultados, se propone fijar $\tau = 0{,}05$ en lugar del umbral por defecto. Con este valor el coste total desciende de 4410 € a 2920 € sobre el conjunto de prueba, una reducción del 34 %. El incremento en falsos positivos (de 1 a 112) es considerable, pero su coste unitario (10 €) es veinte veces inferior al de un falso negativo (200 €), de modo que la operación resulta económicamente ventajosa: se

pasa de 22 FN a 9 FN, y el ahorro por fraudes detectados supera con amplitud el coste adicional por bloqueos erróneos.

Cabe señalar que este umbral se ha determinado exclusivamente a partir de la función de coste definida en la práctica. En un entorno real debería validarse sobre datos de producción y recalibrarse periódicamente si la distribución de transacciones varía.

4.6. Conclusión final

La práctica pone de manifiesto que la elección del umbral de decisión no es una cuestión trivial: fijar $\tau = 0{,}5$ sin análisis previo generó un coste de 4410 €, frente a los 2920 € obtenidos con el umbral óptimo. Hemos verificado que el valor que minimiza el coste total se desplaza hacia la izquierda respecto al punto de equilibrio porque la penalización por FN (200 €) es desproporcionadamente mayor que la de FP (10 €): a medida que se baja el umbral, el ahorro por fraudes adicionales detectados supera el coste extra de los bloqueos erróneos, hasta que ambos efectos se igualan en torno a $\tau^* = 0{,}05$.

Asimismo, se comprueba que en problemas con clases fuertemente desbalanceadas el AUC-ROC puede inducir a conclusiones optimistas, mientras que la curva Precision-Recall refleja con mayor fidelidad la capacidad del modelo para recuperar la clase minoritaria sin disparar el número de falsas alarmas. Ambos resultados subrayan que la evaluación de un clasificador en producción debe incorporar necesariamente los costes asociados al dominio y no limitarse a métricas genéricas.

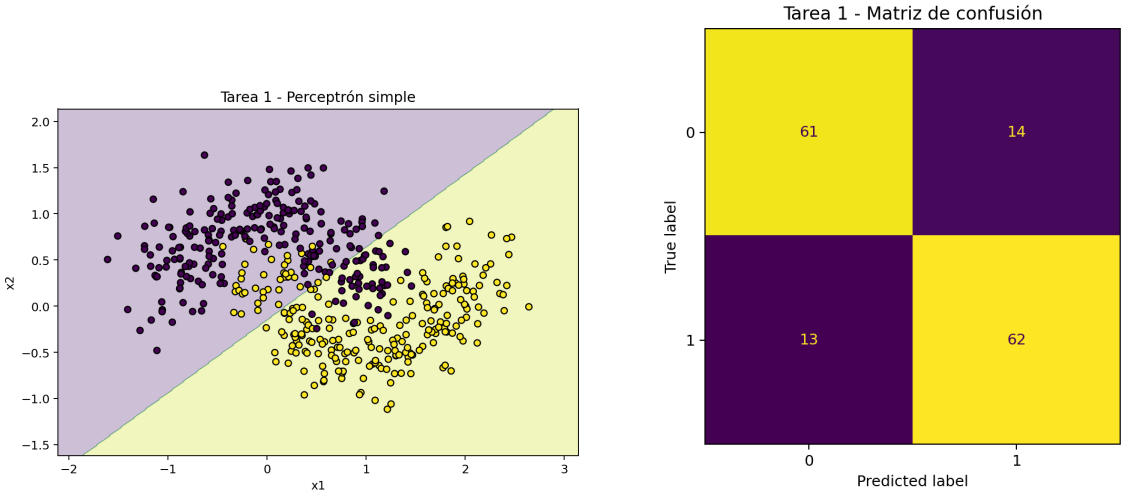
Arquitectura de Redes Neuronales: El Perceptrón Multicapa

En esta práctica se explora el funcionamiento del Perceptrón Multicapa (MLP) y su capacidad para modelar fronteras de decisión no lineales. A diferencia del perceptrón simple, el MLP aborda problemas de clasificación más complejos mediante la introducción de capas ocultas. A lo largo del documento se analizan distintos hiperparámetros, como el número de neuronas y la función de activación, y su impacto en la convergencia y generalización del modelo.

5.1. Tarea 1: La barrera de la linealidad — El Perceptrón Simple

Para iniciar el estudio, se ha evaluado el rendimiento de un Perceptrón simple (un modelo sin capas ocultas, definido con `hidden_layer_sizes=()`) aplicado al dataset sintético `make_moons`. Aunque tras el entrenamiento se alcanza una precisión en test de aproximadamente un 82 %, el análisis visual de las gráficas revela que el modelo falla de manera sistemática en ciertas regiones complejas.

5.1.1. Resultados y Análisis Estructural



(a) Frontera de decisión lineal.

(b) Matriz de confusión.

Figura 5.1: Evaluación del rendimiento en un perceptrón de una sola capa.

Al observar la frontera de decisión (Figura 5.1a), la causa del error se hace evidente. El dataset presenta dos clases distribuidas en forma de lunas entrelazadas, pero el modelo ha trazado un límite de separación completamente recto. Matemáticamente, es imposible separar estas dos clases con un solo hiperplano, lo que explica la distribución de fallos en la matriz de confusión (Figura 5.1b).

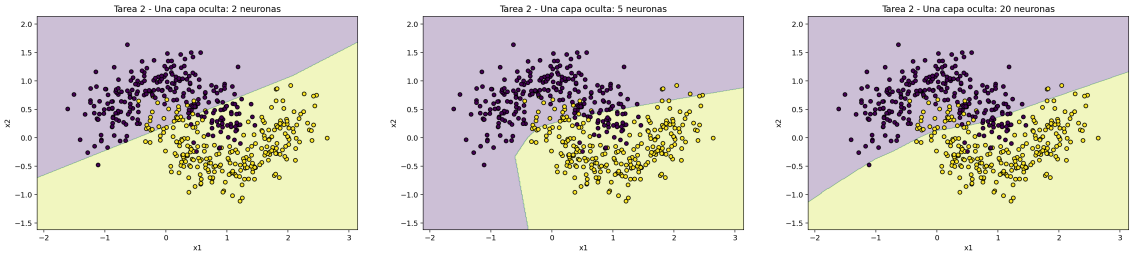
Este comportamiento no indica una falta de entrenamiento, sino una limitación inherente a la arquitectura. Al no disponer de capas ocultas, un perceptrón simple solo puede generar combinaciones lineales de las entradas, restringiendo su capacidad a la creación de fronteras lineales.

Se concluye que el modelo padece un grado severo de *underfitting* debido a su simplicidad. Para clasificar correctamente distribuciones que no son linealmente separables, es necesario dotar a la red de representaciones no lineales mediante el uso de capas ocultas.

5.2. Tarea 2: Impacto de la capacidad en la capa oculta

Una vez constatada la limitación del modelo lineal, se procede a introducir una capa oculta en el MLP para proporcionarle capacidad de decisión no lineal. Se han

evaluado tres configuraciones distintas (2, 5 y 20 neuronas) para ilustrar visualmente la adaptación de la frontera de decisión a medida que aumenta la complejidad de la red.



(a) 2 neuronas: Underfitting. (b) 5 neuronas: Transición. (c) 20 neuronas: Ajuste óptimo.

Figura 5.2: Evolución de la frontera de decisión según la densidad de la red.

Tabla 5.1: Resultados experimentales según la arquitectura.

Arquitectura	**Acc. Test**	**Iteraciones**	**Observación**
MLP(2)	0.8200	648	Capacidad insuficiente
MLP(5)	0.8400	726	Mayor esfuerzo de optimización
MLP(20)	0.8533	406	Convergencia eficiente

Configuración con 2 neuronas

Con dos neuronas, la red adquiere una ligera flexibilidad respecto al límite estático de la tarea anterior. No obstante, al observar la gráfica se aprecia que la frontera forma un ángulo sencillo en forma de pico. La capa carece de los parámetros suficientes para contornear la distribución, manteniendo la red un problema general de *underfitting*.

Configuración con 5 neuronas

Al escalar a 5 neuronas, el MLP consigue envolver ambas clases con mucha mayor efectividad. Resulta evidente en la representación que la frontera no es suavemente curva: al emplear la función de activación ReLU, la forma se delimita ensamblando segmentos de recta por cada neurona oculta. Este comportamiento resuelve las carencias iniciales, dando lugar a un contorno poligonal característico de redes con pocos nodos y activadas por ReLU.

Configuración con 20 neuronas

Finalmente, con 20 neuronas, la convergencia visual es notable, logrando definir una frontera de decisión mucho más precisa que se adapta al contorno de ambas clases sin ser excesivamente irregular. A pesar de incrementar la capacidad de la red, el modelo no incurre en *overfitting*, ya que evita adaptarse al ruido estadístico de los datos. Esto se confirma al observar que las métricas de *train* (86.00 %) y *test* (85.33 %) son prácticamente idénticas, mostrando una buena capacidad de generalización, aunque la mejora respecto a la configuración de 5 neuronas es marginal.

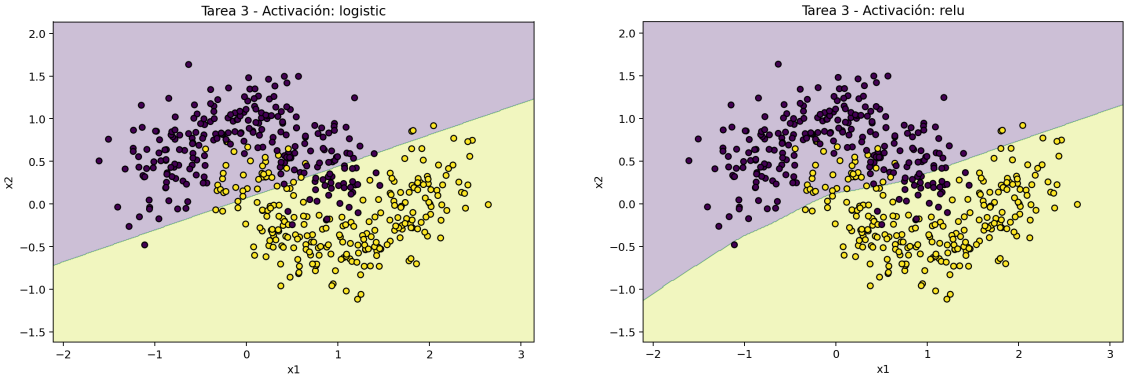
Conclusión

Se confirma visualmente que añadir neuronas a la capa oculta permite superar la limitación del perceptrón simple, dotando al modelo de la capacidad para trazar fronteras de decisión complejas (en este caso, aproximando curvas mediante la superposición de segmentos rectos generados por la activación ReLU). Este comportamiento ilustra el compromiso fundamental en el diseño de arquitecturas: la red debe tener suficientes neuronas para evitar el *underfitting*, pero sin sobredimensionarse, para no penalizar el coste computacional a cambio de mejoras insignificantes en la precisión.

5.3. Tarea 3: Dinámica de Aprendizaje — ReLU vs. Sigmoide

Manteniendo una arquitectura uniforme basada en una capa oculta de 20 neuronas, se expone ahora la comparativa entre la función logística (*Sigmoide*) y la unidad lineal rectificada (*ReLU*). El fin de este ejercicio es medir la eficacia en la separación y los tiempos de entrenamiento modificando únicamente el selector de activación de la red.

5.3.1. Fronteras de decisión



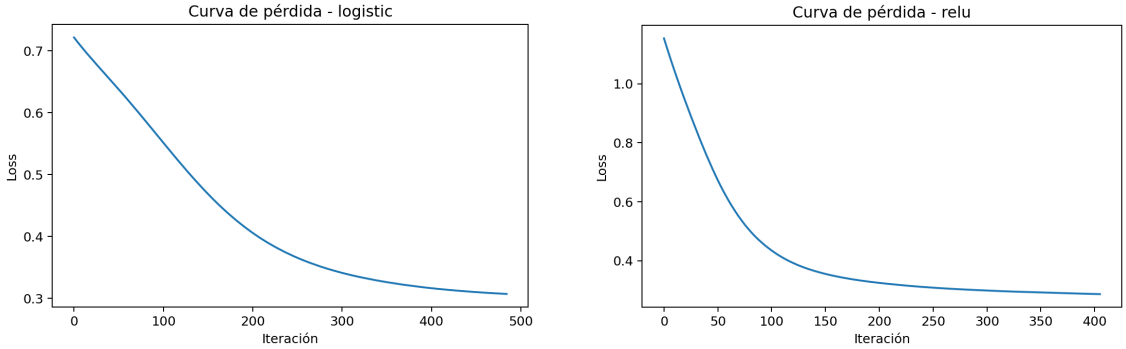
(a) Frontera con Sigmoide.

(b) Frontera con ReLU.

Figura 5.3: Comparación visual de la frontera de decisión según la activación.

Ambas funciones resuelven de forma adecuada la separación no lineal, aunque exhiben trazados característicos acordes a sus formulaciones algebraicas. La función Sigmoide genera una frontera de decisión suave y continua debido a su naturaleza asintótica en el intervalo (0, 1). Por el contrario, la activación ReLU produce un límite definido por la intersección de segmentos rectos, generando contornos más abruptos y poligonales.

5.3.2. Curvas de pérdida y velocidad de convergencia



(a) Curva de pérdida con Sigmoide.

(b) Curva de pérdida con ReLU.

Figura 5.4: Comparativa de la velocidad de aprendizaje.

Tabla 5.2: Métricas comparativas por función de activación (arquitectura `(20,)`).

Activación	Acc. Train	Acc. Test	Iteraciones	Loss final
Sigmoide (logistic)	0.8657	0.8400	485	0.3071
ReLU	0.8600	0.8533	406	0.2870

Análisis

El comportamiento visualizado en la Figura 5.4 se corrobora con las métricas (Tabla 5.2). Al finalizar el entrenamiento, el modelo con ReLU converge en 406 iteraciones, aventajando claramente a su homólogo con función Sigmoide, que necesita 485 iteraciones para alcanzar el mismo criterio de parada.

Esta diferencia de rendimiento se debe al problema del desvanecimiento del gradiente (*vanishing gradient*). La función Sigmoide satura sus salidas en los extremos (0 y 1), lo que provoca que su derivada tienda a cero. Durante el *backpropagation*, al multiplicar estos gradientes casi nulos, la actualización de los pesos en las primeras capas se paraliza, ralentizando la convergencia. Por el contrario, la función ReLU tiene una derivada constante de 1 para cualquier valor positivo. Esto permite que el error fluya hacia atrás sin atenuarse, evitando la congelación de los coeficientes y logrando una convergencia mucho más rápida (406 iteraciones frente a 485).

Aunque en este problema bidimensional sencillo ambas funciones logran resultados aceptables (entorno al 84-85 %), en arquitecturas multicapa más profundas la saturación de la Sigmoide provoca caídas críticas de rendimiento, siendo esta la razón por la que ReLU es el estándar actual.

5.4. Tarea 4: Reconocimiento de Dígitos y Simplicidad de la Arquitectura

Como prueba final de diseño de un MLP, el objetivo es alcanzar una precisión superior al 95 % en la clasificación del dataset de dígitos escritos a mano (`load_digits`), manteniendo la arquitectura de la red lo más simple posible. Para ello, se han evaluado seis configuraciones diferentes, variando la profundidad (número de capas) y la anchura (neuronas por capa), todas ellas entrenadas utilizando el optimizador Adam.

Tabla 5.3: Métricas obtenidas para distintas arquitecturas en el dataset de dígitos.

Arquitectura	Acc. Train	Acc. Test	Iteraciones	Profundidad	Meta
MLP(20)	1.0000	0.9644	276	1 capa	✓
MLP(50)	1.0000	0.9711	180	1 capa	✓
MLP(100)	1.0000	0.9800	147	1 capa	✓
MLP(30, 15)	1.0000	0.9600	161	2 capas	✓
MLP(50, 20)	1.0000	0.9756	131	2 capas	✓
MLP(64, 32)	1.0000	0.9667	110	2 capas	✓

5.4.1. Análisis de sobreajuste

Al analizar la Tabla 5.3, se observa que todas las arquitecturas alcanzan un 100 % de precisión en entrenamiento (`acc_train` = 1.0000). Esta asimetría frente a los resultados de test indica un claro sobreajuste (*overfitting*) inicial. Es un comportamiento previsible: estamos aplicando redes con alta capacidad de representación sobre un conjunto de datos relativamente pequeño (aprox. 1.350 muestras de validación cruzada).

A pesar de memorizar el conjunto de entrenamiento, el modelo no pierde su capacidad de generalización (manteniendo un 96-98 % en test). Esto se logra gracias al término de regularización L2 implementado por defecto en Scikit-Learn (`alpha=1e-4`), que penaliza el crecimiento descontrolado de los pesos y evita que la red se ajuste al ruido estadístico.

Cabe destacar el papel crítico de este hiperparámetro en el compromiso entre capacidad y generalización. Si eliminásemos esta regularización (`alpha=0`), la frontera de decisión se volvería excesivamente irregular, adaptándose a cada punto de ruido y desplomando la precisión en test. Por el contrario, si aplicásemos una penalización demasiado estricta (ej. `alpha=10`), forzaríamos los pesos a valores cercanos a cero. Esto limitaría artificialmente la capacidad expresiva de la red y provocaría un *underfitting* severo, devolviéndonos a un escenario de fallo similar al observado con el perceptrón simple en la Tarea 1.

5.4.2. Elección del modelo

Al evaluar la topología a partir de los datos de la Tabla 5.3, surge el debate clásico: ¿es preferible ensanchar una sola capa o añadir profundidad con una segunda capa? En problemas complejos de visión artificial, extraer características jerárquicas con redes profundas es vital. Sin embargo, para este dataset concreto (`load_digits`, imágenes de 8x8 píxeles), añadir una segunda capa oculta (como en MLP(30, 15) o

MLP(64, 32)) no aporta una ventaja competitiva real frente a ensanchar una sola. De hecho, las topologías de dos capas logran precisiones similares (entorno al 96-97 %) pero complican la arquitectura de forma innecesaria. Para este nivel de complejidad, la experimentación demuestra que una sola capa ancha es suficiente para capturar las no linealidades sin dificultar el entrenamiento.

Aunque el modelo MLP(100) alcanza la precisión más alta (98.00 %), bajo un criterio de ingeniería eficiente la opción óptima es el modelo MLP(20). Esta arquitectura más simple supera holgadamente el requisito del 95 % de precisión planteado en la práctica, pero siendo mucho más ligera computacionalmente. Para ponerlo en perspectiva, el clasificador MLP(20) solo necesita optimizar alrededor de 1.500 parámetros frente a los más de 7.500 que exige la arquitectura de 100 neuronas, logrando una eficacia prácticamente idéntica con una quinta parte del coste estructural.

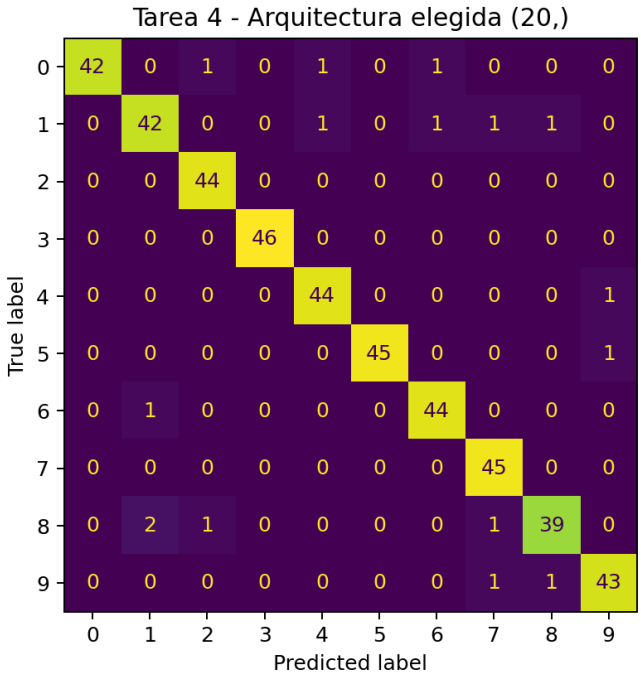


Figura 5.5: Matriz de confusión de la arquitectura **MLP(20)**. El desempeño es casi impecable en todas las clases; los escasos errores se concentran en pares de dígitos visualmente similares.

1.5. Propuesta Técnica de Despliegue

Finalizada la validación experimental, se exponen las decisiones de diseño recomendadas para la integración del modelo clasificador en un entorno de producción:

1. Arquitectura recomendada La configuración idónea es el modelo **MLP(20)**, compuesto por una única capa oculta de 20 neuronas y activado mediante la función **ReLU**. Se descarta el uso de un modelo lineal (cero capas ocultas) porque es estructuralmente incapaz de resolver fronteras de decisión complejas. La función ReLU se establece como el estándar preferido frente a la Sigmoide, ya que mitiga el desvanecimiento del gradiente y logra una convergencia más estable. Con esta configuración, el sistema alcanza un 96.44 % de precisión, superando el objetivo técnico del 95 %.

2. Justificación de simplicidad vs. profundidad Aunque el incremento a 100 neuronas mejora la precisión ligeramente (hasta un ∼98 %), supone multiplicar por cinco la cantidad de parámetros de la red. En un entorno real, esta ganancia marginal no compensa el sobrecoste computacional. Mantener la topología en 20 neuronas proporciona latencias más bajas y un menor consumo de memoria, facilitando su posible ejecución en arquitecturas de recursos limitados (*Edge Computing*). Además, al limitar la capacidad de la red, se reduce el riesgo de *overfitting*, haciendo que el modelo sea más robusto frente a variaciones en los datos de producción (*Data Drift*).

3. Estrategia de monitorización Debido a la tendencia inicial de la red a memorizar el conjunto de entrenamiento, se recomienda desplegar el modelo inicialmente bajo la modalidad de *shadow deployment* (despliegue en sombra). Esto permitirá evaluar la fiabilidad de sus predicciones frente al tráfico de datos real en tiempo de ejecución, pero sin impactar los procesos operativos hasta confirmar su total estabilización.

Cuaderno de Laboratorio — Práctica 7: La Inteligencia Colectiva (Modelos de Ensambles)

Introducción

En esta práctica se aborda un problema de clasificación binaria utilizando el conjunto de datos *Breast Cancer Wisconsin*, el cual contiene características extraídas de imágenes digitales de aspiraciones con aguja fina de masas mamarias. El objetivo principal es diagnosticar tumores como benignos o malignos empleando tres enfoques distintos: un Árbol de Decisión solitario, Random Forest (Bagging) y Gradient Boosting.

6.1. Tarea 1 — Árbol Solitario (Baseline)

En primer lugar, se evaluó un Árbol de Decisión estándar sin límite de profundidad (`max_depth=None`).

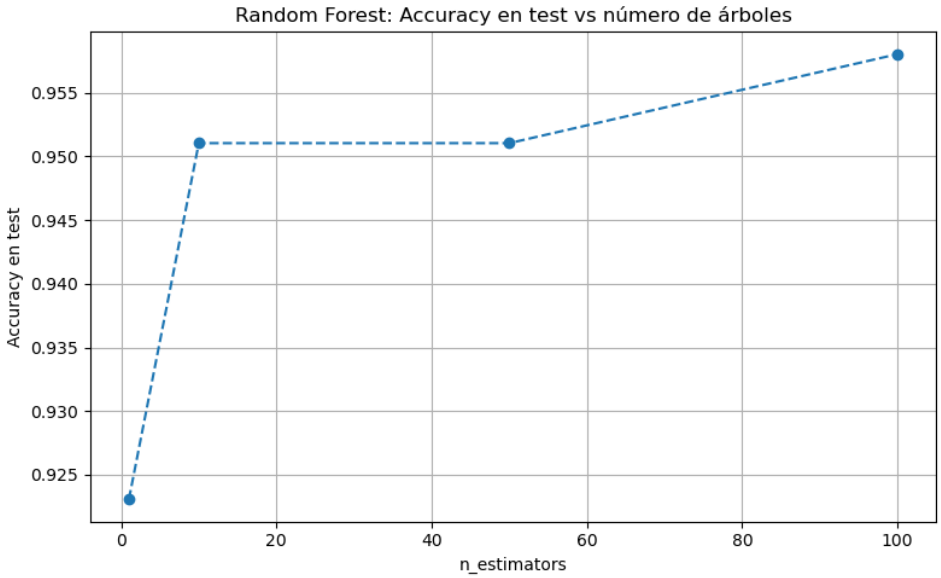
Resultados obtenidos:

- *Accuracy* en entrenamiento: 1.0000
- *Accuracy* en test: 0.9231

Análisis de Sobreajuste: La brecha entre el rendimiento perfecto en entrenamiento (100 %) y la caída en test (92.31 %) es un claro síntoma de sobreajuste (*overfitting*). Al no limitar la profundidad del árbol, el algoritmo continúa dividiendo el espacio de características hasta crear particiones diminutas (nodos hoja puros) que contienen incluso una única muestra. Esto provoca una extrema sensibilidad al ruido inherente de los datos de entrenamiento. En consecuencia, el modelo no solo "memoriza" el conjunto de datos, sino que pierde drásticamente su capacidad de generalización frente a instancias no vistas, ya que modela excepciones en lugar de patrones reales de la distribución subyacente.

6.2. Tarea 2 — Bagging y Random Forest

Para mitigar los problemas de alta varianza del árbol simple, se aplicó un ensamblado de tipo *Bagging* mediante Random Forest, evaluando el rendimiento en función de la cantidad de estimadores (árboles).



Análisis de Generalización y Saturación: La figura ilustra cómo el *accuracy* en el conjunto de test mejora drásticamente de 0.923 (1 árbol) a 0.958 (100 árboles). Esta mejora se debe a que Random Forest entrena múltiples árboles descorrelacionados sobre submuestras aleatorias (*bootstrap*) y promedia sus predicciones (consenso). Esto reduce significativamente la varianza global sin incrementar el sesgo, mitigando el sobreajuste. Se observa, además, que la curva de rendimiento se satura a partir de los 50 árboles (estabilizándose en torno al 0.951 – 0.958). Esto ocurre porque el ensamble alcanza un punto donde añadir más árboles no aporta diversidad adicional significativa a las votaciones, incurriendo únicamente en mayor coste computacional sin apenas mejoras en el error de generalización.

6.3. Tarea 3 — El poder del Boosting

Posteriormente, se evaluó el algoritmo Gradient Boosting.

Paralelo (RF) vs Secuencial (Boosting): A diferencia de Random Forest, que entrena árboles independientes de forma estrictamente paralela, Gradient Boosting construye los árboles de manera secuencial. Cada nuevo estimador se entrena

para corregir los errores residuales (gradiente de la función de pérdida) cometidos por el conjunto de árboles anteriores.

Rendimiento y Coste:

- *Accuracy* en test: 0.9580
- Tiempo de entrenamiento: $\approx$ 0.2512 segundos.

Gradient Boosting igualó el excelente rendimiento de Random Forest en test (95.8 %), pero con un proceso de entrenamiento iterativo que, por su naturaleza secuencial, no se beneficia de la misma paralelización, resultando en un tiempo de ejecución ligeramente superior o que exige mayor cuidado paramétrico.

Importancia de Variables: Las 3 variables más determinantes identificadas por el modelo fueron:

1. `worst radius` (importancia: $\approx$ 55.9 %)
2. `worst concave points` (importancia: $\approx$ 13.2 %)
3. `worst perimeter` (importancia: $\approx$ 12.8 %)

Interpretación Clínica: Estas características reflejan las dimensiones y la morfología del perímetro del núcleo celular. Las células malignas suelen presentar deformidades estructurales acusadas (radios amplios y perímetros irregulares) y una gran cantidad de puntos cóncavos en su contorno. La capacidad del modelo para distinguir estas anomalías explica su alto poder discriminativo en el diagnóstico oncológico.

6.4. Tarea 4 — Comparación Global y Reflexión

Tabla 6.1: Comparativa global de los modelos evaluados.

Modelo	Acc. Test	Coste	Ventajas	Desventajas
Árbol Simple	0.923	Muy bajo	Máxima interpretabilidad, inferencia ultrarrápida.	Fuerte sobreajuste, alta varianza.
Random Forest	0.958	Medio	Paralelizable, reduce varianza, robusto.	Pérdida de interpretabilidad (caja negra).
Gradient Boosting	0.958	Alto	Altísima precisión, corrige sesgo progresivamente.	Coste computacional alto, propenso a sobreajuste si no se regula bien.

Inconvenientes de los ensambles: A pesar de su excelente rendimiento, los ensambles presentan dos problemas fundamentales. En primer lugar, se convierten en modelos de *caja negra*, donde seguir la lógica de decisión de cientos de árboles simultáneos es inabarcable para un humano, perdiendo interpretabilidad. En segundo lugar, multiplican los requerimientos de memoria y CPU tanto en la fase de entrenamiento como durante la inferencia, dificultando su uso en entornos con recursos limitados.

Reto de Despliegue:
Si tuvieras que desplegar un modelo en un dispositivo con CPU y memoria limitados, ¿cuál elegirías y por qué?

Para un despliegue en un dispositivo muy limitado (ej. un *wearable* o un sistema *embedded* básico), elegiría el **Árbol Simple** o, como alternativa de compromiso, un **Random Forest extremadamente reducido** (de 1 a 10 estimadores). Al seleccionar un único Árbol de Decisión, **sacrifico** un porcentaje de precisión de aproximadamente un 3.5 % y la robustez frente al ruido de nuevas muestras. Sin embargo, **gano** un modelo con un consumo de memoria insignificante (unos pocos kilobytes para almacenar las reglas de partición) y un coste de inferencia mínimo equivalente a una docena de evaluaciones condicionales `if-else`. En sistemas críticos donde la latencia o la batería dictan la viabilidad del proyecto, esta enorme reducción del uso computacional justifica plenamente la pérdida marginal de *accuracy*.

Bibliografía